

A Project Report

Submitted To

The Department of Computer Science and Engineering, Metropolitan University, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

On

Sarabo: An Electronics Repair Service Management Platform

Submitted By

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)

Under The Supervision of

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

Submission Date: 26 October 2026



STUDENT'S DECLARATION

I, the undersigned, solemnly declare that the project report entitled **"Sarabo: An Electronics Repair Service Management Platform"** is based on my original project work carried out during the course of study under the supervision of **Shrabanti Chowdhury**. The analysis, design, implementation, testing, and documentation presented in this report were completed as an individual final year project, except where the work of others has been properly cited.

I further declare that, to the best of my knowledge and belief, this report has not been submitted elsewhere in substantially the same form for the requirement of any other degree. All external literature, standards, documentation, libraries, and technologies used in the project have been acknowledged in the reference section.

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)



SUPERVISOR'S RECOMMENDATION

The project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), is a record of project work carried out under my supervision. I hereby recommend that this project report is satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering and may be processed for further evaluation.

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

(Supervisor)



LETTER OF APPROVAL

This is to certify that the project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), has been studied and approved by the project examination committee as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Shrabanti Chowdhury

Senior Lecturer, Department of Computer Science and Engineering

Supervisor

Nabila Zannat Rifa

Senior Lecturer

Reviewer 1

Md. Fahmidur Rahman Sakib

Senior Lecturer

Reviewer 2

Abstract

Electronics repair in Bangladesh is largely informal: customers hand over a device without knowing the price or its progress, and independent technicians have no structured way to find work or get paid. This project presents **Sarabo**, a web platform that turns repair into a tracked, priced and auditable service for three roles. A *Customer* describes the problem (optionally with AI help in Bangla, English or Banglish), books a pickup window, compares ranked offers from qualified local technicians, pays an inspection fee that counts toward the final price, approves an itemised quote, pays, tracks the repair and is covered by a 30-day warranty. A *Technician* finds work on an open jobs board, sends offers, inspects, quotes, repairs and withdraws earnings through bKash. An *Admin* approves technicians, manages a 45-service catalogue, resolves refunds, disputes and stalled work, and reads reports with an AI summary.

Sarabo is built with React 19, Node.js/Express 5 and MongoDB, using Firebase Authentication, Stripe (test mode), Supabase storage and Google Gemini, and is deployed on Vercel. Every money- or state-changing step is checked on the server and runs in a database transaction, and every waiting state has a deadline or an admin alert. The system passes 2,761 automated server checks and 58 browser tests in continuous integration.

Keywords: Electronics repair, service marketplace, workflow management, React, Express, MongoDB, Stripe, generative AI.

Acknowledgements

I express my sincere gratitude to my supervisor, **Shrabanti Chowdhury**, Senior Lecturer, Department of Computer Science and Engineering, Metropolitan University, for her guidance, feedback, and encouragement throughout this final year project. Her supervision helped me refine Sarabo from an initial application idea into a structured service management platform.

I am also grateful to the faculty members of the Department of Computer Science and Engineering for the academic foundation that supported this work, and to the reviewers, **Nabila Zannat Rifa** and **Md. Fahmidur Rahman Sakib**, for evaluating the project and for their feedback.

Finally, I acknowledge the open-source communities and documentation maintainers behind React, Express, MongoDB, Firebase, Stripe, Supabase, Google Gemini, TanStack Query, Tailwind CSS, Playwright and the other tools used in Sarabo. Their work made it possible to build and test a full-stack platform of this scope as an individual project.

Contents

Abstract I
Acknowledgements II
Table of Contents III
List of Tables VII
List of Figures VIII
Listings IX

1 Introduction **1**
1.1 Project Overview 1
1.2 Problem Statement 2
1.3 Motivation 2
1.4 Objectives 3
1.5 Scope 3
1.6 Individual Contribution 4
1.7 Report Organization 4

2 Background and Related Work **5**
2.1 Web-Based Service Management 5
2.2 Single-Page Application Architecture 5
2.3 RESTful API and Express 5
2.4 Document Database and Transactions 6
2.5 Authentication and Authorization 6
2.6 Online Payment Processing 6
2.7 Private Object Storage 6
2.8 Large Language Models in Applications 7
2.9 Usability and Accessibility 7
2.10 Related Systems 7

3 Requirements Analysis **9**
3.1 Stakeholders and Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 11
3.4 Service Catalogue 11
3.5 Use-Case View 12

4 System Design and Architecture **14**
4.1 High-Level Architecture . . . 14
4.2 Client Architecture . . . 14
4.3 Server Architecture . . . 15
4.4 Data Collections . . . 15
4.5 Repair Request Document . . . 15
4.6 State Model . . . 17
4.7 The Service Spine . . . 17
4.8 API Design . . . 18

5 Implementation **19**
5.1 Technology Stack . . . 19
5.2 Development Process . . . 20
5.3 Frontend Implementation . . . 21
5.4 Backend Implementation . . . 22
5.5 Service Catalogue and Matching . . . 22
5.6 Pickup Scheduling . . . 23
5.7 Quotation Model . . . 23
5.8 Payment Implementation . . . 23
5.9 Technician Settlement and Wallet . . . 23
5.10 Damage and Evidence Images . . . 24
5.11 Public Tracking and Jobs Board . . . 24
5.12 Deployment and Continuous Integration . . . 24
5.13 Final Improvement Phase . . . 25

6 Core Workflows and Business Rules **27**
6.1 End-to-End Repair Workflow . . . 27
6.2 Request Creation . . . 27
6.3 Job Portal and Offers . . . 27
6.4 Offer Ranking (Smarter Matching) . . . 29
6.5 Inspection Fee and Booking . . . 29
6.6 Inspection, Quotation and Revision . . . 30
6.7 Payment, Repair and Receipt . . . 30
6.8 No Request Can Get Stuck . . . 31
6.9 Repair Warranty . . . 32
6.10 Settlement and Withdrawal Lifecycle . . . 33
6.11 Cancellation Rules . . . 34
6.12 Technician Profile Editing . . . 34
6.13 Service-Area Map . . . 34
6.14 Feedback and Moderation . . . 34
6.15 Notifications and Reminders . . . 35
6.16 Damage Image Upload . . . 35

7 System Walkthrough **36**
7.1 Public Website . . . 36
7.2 Request and Offer . . . 38
7.3 Choosing and Booking . . . 40
7.4 Pickup, Inspection and Quote . . . 41
7.5 Payment, Repair and Hand-Over . . . 43
7.6 Warranty . . . 44
7.7 Technician and Admin . . . 45

8 AI-Assisted Features **47**
8.1 AI Customer Support Chat . . . 47
8.2 AI Request Helper . . . 47
8.3 Technician Price Guide and AI Offer Note . . . 48
8.4 AI Summary of Admin Reports . . . 48
8.5 Safeguards . . . 48

9 Security, Reliability and Data Integrity **50**
9.1 Security Principles . . . 50
9.2 Authentication . . . 50
9.3 Role- and State-Based Authorization . . . 50
9.4 Financial Integrity . . . 51
9.5 Privacy . . . 51
9.6 Private Image Security . . . 51
9.7 Input Validation and Safe Errors . . . 51
9.8 CORS, Rate Limiting and Abuse Control . . . 52
9.9 Transaction Safety . . . 52
9.10 Environment Isolation . . . 52
9.11 Legacy Compatibility Boundary . . . 52

10 Testing and Evaluation **54**
10.1 Testing Strategy . . . 54
10.2 Verification Results . . . 55
10.3 Representative Test Scenarios . . . 56
10.4 Manual End-to-End Acceptance . . . 57
10.5 Evaluation Against Objectives . . . 58
10.6 Validation Boundary . . . 58

11 Limitations and Future Work **59**
11.1 Current Limitations . . . 59
11.2 Future Enhancements . . . 60

12 Conclusion **61**

References **61**

Appendices **64**

A API Surface **65**

B Data Model Summary **68**

C Financial Rules **69**

D Source and Build Notes **70**

E Incident Record: Revised Quote Hang **72**

List of Tables

1.1 Project objectives 3

2.1 Comparison of approaches to electronics repair 8

3.1 User roles 9
3.2 Functional requirements 10
3.3 Non-functional requirements 12
3.4 Service catalogue summary with sample services 12

4.1 MongoDB collections 16
4.2 Repair lifecycle states 17
4.3 Representative API endpoints 18

5.1 Implemented technology stack 19
5.2 Development phases (from the git history) 20
5.3 Final improvement phase log 25

6.1 Offer ranking factors 29
6.2 Inspection fee outcomes 30
6.3 Situations that used to block a request, and how they end now 31

8.1 AI risks and safeguards 49
8.2 Default AI usage limits 49

10.1 Verification results (14–16 October 2026) 55
10.2 Representative test scenarios 56
10.3 Evaluation against objectives 58

A.1 Sarabo API endpoints (main families) 65

B.1 Selected fields of a repair request 68

C.1 Worked example 69

D.1 Verified code baseline 70

List of Figures

3.1 Role-oriented use-case view 13

4.1 Sarabo high-level architecture 14
4.2 Conceptual structure of a V2 repair request 16
4.3 The four-stage service spine shown to users 17

6.1 End-to-end repair workflow 28
6.2 Rejected-quote recovery 31
6.3 Warranty claim lifecycle 32
6.4 Technician settlement and withdrawal lifecycle 33
6.5 Damage-image upload workflow 35

7.1 Service areas: searching "Ambarkhana" zooms the map to that part of Sylhet (Section 6.13). 37
7.2 AI support chat: the price and the 200 BDT fee come from the live catalogue (Chapter 8). 37
7.3 Public jobs board: a safe summary of each open job, with no customer details. . . 38
7.4 AI request helper: the Banglish text became the device, the catalogue service and an English description. 38
7.5 Address and a two-hour pickup window (Friday closed). 39
7.6 Technician's offer: price guide, minimum fee, and a note written by AI from a few words. 39
7.7 The offer as the customer sees it, with its ranking reasons (Section 6.4). 40
7.8 Choosing the offer: the inspection fee is paid by Stripe (test mode) to book the visit. 40
7.9 The device's condition and accessories recorded at pickup. 41
7.10 Inspection: findings, issues and a repairability decision. 41
7.11 Itemised quote (800 + 3,200 = 4,000 BDT), with the 10% commission shown. . . 42
7.12 The customer's view of the quote: nothing is charged before approval. 42
7.13 Payment successful (3,800 BDT after the 200 BDT fee), with a PDF receipt. . . . 43
7.14 Completing the repair; "Can't finish this repair" would send it to an admin. 43
7.15 Confirm receipt, or "I haven't received it"; confirmed automatically after 3 days. . 44
7.16 Reporting a problem under the 30-day warranty. 44
7.17 The resolved claim: free re-inspection, "Covered", re-repair and the customer's "Working now". 45
7.18 Technician wallet: the 3,600 BDT earning requested as a bKash withdrawal. . . . 45

7.19 Admin overview: missed pickups, requests without offers and stalled repairs come first. 46
7.20 Admin reports with the AI summary; sentences with numbers not in the data are removed. 46
7.21 Recording a bKash payout with its TrxID (a test value). 46

10.1 Testing layers used in Sarabo 54

Listings

5.1 Quote total and amount still to pay 23
5.2 Settlement rule (whole taka) 24
D.1 Development commands 70
E.1 Round-aware quote notification key 72

Chapter 1

Introduction

1.1 Project Overview

Sarabo is a web-based electronics repair service management platform developed as an individual final year project. It coordinates the complete life of a repair request (from the moment a customer describes a faulty device to the moment the technician is paid) through one traceable workflow. The platform connects three actors: a *Customer* who needs a repair, a *Technician* who performs it, and an *Admin* who keeps the service running.

The central design decision is that a repair request is a *stateful service case*, not a simple database record. Every important activity (request creation, technician offers, booking, pickup, inspection, quotation, payment, repair, handover, settlement and feedback) moves that case from one well-defined state to another, and the server checks that each move is allowed for that user at that moment. This is what separates Sarabo from a basic create-read-update-delete (CRUD) application: the correctness of a later action depends on earlier actions having happened, and having happened only once.

The project went through two major stages. The first stage (July–August 2026) built the core repair lifecycle with admin-driven technician assignment, and was described in an earlier version of this report. The second stage (September–October 2026) turned Sarabo into a two-sided marketplace: an open jobs board where technicians send offers, a paid inspection fee with automatic refunds, pickup scheduling, customer reviews, email notifications, operational reports, four AI-assisted features and a production deployment. A final phase (11–16 October 2026) added offer

ranking, a repair warranty, deadlines for every waiting state, a minimum inspection fee and faster hosting. This report describes the system as it stands after all of this.

1.2 Problem Statement

A repair service involves both technical and administrative coordination. A customer needs to describe the device and problem, understand the likely cost before handing the device over, choose someone trustworthy, pay securely and know whether the device has been repaired. A technician needs a steady source of nearby jobs that match their skills, a structured way to record an inspection and quote, and a reliable way to be paid. An administrator needs to see which requests are stuck, which payments need attention, and whether demand is being turned away.

When these activities are handled through phone calls, messaging apps and paper records, inconsistent states easily occur: a customer is charged before agreeing to a price, a technician is booked twice for the same time, an inspection fee is neither refunded nor credited, a technician's earning is paid twice, or private photos of a customer's device are exposed. Sarabo addresses this by modelling the repair process as a role-aware workflow with explicit transition rules, server-side validation and transactional updates.

1.3 Motivation

The project was motivated by five goals:

1. to address a real, local problem: making electronics repair in Bangladesh more transparent and trustworthy for customers and more accessible to independent technicians;
2. to build a realistic full-stack system that goes beyond simple CRUD operations, where roles and state transitions are enforced on the server rather than trusted to the browser;
3. to integrate production-grade external services (identity, card payment and refunds, private object storage, email, generative AI) without exposing privileged credentials;
4. to apply generative AI where it genuinely helps users, while keeping it safe: no invented prices, no leaked personal data, and hard usage limits; and

5. to give every user a clear interface in which the next meaningful action is always visible.

1.4 Objectives

The main objective is to develop a reliable electronics repair management platform that manages the complete service lifecycle. The specific objectives are listed in Table 1.1.

Table 1.1: Project objectives

No.	Objective
O1	Provide secure authentication and role-aware access for Customer, Technician and Admin.
O2	Let customers create structured repair requests (device, service, region, pickup window, optional damage photos), with optional AI assistance.
O3	Match requests only to approved technicians with the right expertise in the customer's region, through an open offer-based job portal.
O4	Support a paid inspection fee that books a technician, is credited toward the final price, and is refunded or split fairly when the repair does not go ahead.
O5	Provide an inspection and quotation workflow with customer approval, rejection and revision.
O6	Accept payment only for approved amounts, with a single authoritative paid state and downloadable receipts.
O7	Support repair progress, completion evidence, receipt confirmation and customer feedback.
O8	Keep customers and technicians informed through in-app notifications, email, reminders and a sanitized public tracking page.
O9	Provide a technician wallet with a 90/10 settlement and controlled bKash withdrawals.
O10	Give admins operational oversight: attention lists, reports, catalogue management and moderation.
O11	Validate the system through automated server tests, browser tests, static checks and continuous integration, and deploy it.
O12	Protect customers after the repair with a warranty, and make sure no request can get stuck in any state.

1.5 Scope

The implemented scope includes registration and email verification, technician applications with structured expertise, a versioned service catalogue, AI-assisted request creation, damage-photo upload, pickup scheduling, an open jobs board with offers, inspection-fee booking and refunds, inspection, quotation and revision, Stripe payment, repair progress and completion evidence,

receipt confirmation, reviews and private reports, in-app and email notifications, public tracking, technician wallet and bKash withdrawals, an AI customer-support chat, admin reports with an AI summary, offer ranking, a 30-day repair warranty, deadlines and admin alerts for stalled work, device-condition records at pickup, and production deployment.

The project does not try to implement every capability of a commercial field-service company. Automated mobile-wallet payouts, cash payment, spare-parts inventory, live GPS tracking of technicians, native mobile apps and multi-branch accounting are outside the current scope and are discussed as future work in Chapter 11.

1.6 Individual Contribution

Sarabo was completed as a solo project by Jahid Hasan. The author was responsible for requirements analysis, data modelling, user-interface design, frontend and backend implementation, workflow and financial logic, integration of external services, AI prompt and safety design, testing, deployment, debugging and the preparation of this report.

1.7 Report Organization

Chapter 2 introduces the technical background and related systems. Chapter 3 defines stakeholders and requirements. Chapter 4 presents the architecture, data model and state model. Chapter 5 describes the implementation and the development process. Chapter 6 explains the core workflows and business rules. Chapter 7 walks through the running system with screenshots of every main step for each role. Chapter 8 covers the AI-assisted features and their safeguards. Chapter 9 discusses security, reliability and data integrity. Chapter 10 presents testing and evaluation. Chapter 11 lists limitations and future work, and Chapter 12 concludes the report. The appendices summarise the API, the data model, the financial rules, build notes and a record of notable incidents.

Chapter 2

Background and Related Work

2.1 Web-Based Service Management

Unlike a static information system, a service platform moves each request through decisions made by different people. Sarabo uses a resource-oriented HTTP API in the REST style [1]: the browser presents the workflow, but only the server decides whether a transition is legal.

2.2 Single-Page Application Architecture

The client is a React single-page application: React composes stateful components [2], React Router maps public and role-specific routes, and TanStack Query manages server state, caching and invalidation [3]. This matters because request status, offers, quotes, balances and notifications change independently of the form a user is filling in. Vite [4] builds the app and Tailwind CSS [5] styles it.

2.3 RESTful API and Express

The server uses Express 5 [6], with routes grouped by domain (users, technicians, requests, offers, inspection, quotes, payments, repairs, warranty, wallet, feedback, notifications, tracking, catalogue, uploads, support, administration). Sensitive routes check authentication and role before any database operation.

2.4 Document Database and Transactions

MongoDB [7] suits a repair request, which keeps one identity while gathering nested workflow data (product and price snapshots, images, inspection, quote and history, payment, evidence, assignment history, settlement). Separate collections hold things with their own lifecycle, index or audit trail (offers, payments, notifications, tracking events, withdrawals, reviews, warranty claims), and multi-document transactions on a replica set change several of them together or not at all.

2.5 Authentication and Authorization

Firebase Authentication supplies identity and signed ID tokens [8]; the server verifies each token with the Firebase Admin SDK before trusting the caller's email. Authorization is role-based (Customer, Technician, Admin) and state-based (the request must be in the right stage). The stored Technician role is still `rider`, from the original codebase, but users only see "Technician".

2.6 Online Payment Processing

Card payments use Stripe Checkout [9], and the server's record is authoritative: Stripe calls a webhook verified with a shared secret [10], so a payment is recorded even if the customer closes the browser. Refunds use idempotency keys, so none is issued twice. Payments currently run in test mode.

2.7 Private Object Storage

Images live in a private Supabase Storage bucket [11]. The browser uploads directly through a short-lived signed URL, the server verifies the stored file's real type and size, and every view uses a fresh signed URL; large files never pass through the API, and the storage key never reaches the browser.

2.8 Large Language Models in Applications

Large language models such as Google Gemini [12] turn free text into structured data, summarise and draft messages, but can also be confidently wrong, be steered by instructions hidden in user text (prompt injection), and cost unpredictably; OWASP lists prompt injection and overreliance among the main risks [13]. Sarabo therefore uses AI only as an *assistant*: answers follow a JSON schema, are checked against live data, never set a price, and are reviewed by a person (Chapter 8).

2.9 Usability and Accessibility

The interface is action-first: the repair's stage and the role's next action outweigh secondary data, and a "service spine" (Request, Inspect, Approve, Repaired) recurs on the home page, tracking page and dashboards. Labels, focus order, keyboard access, contrast, reduced motion and layouts from 320 px follow WCAG 2.2 [14].

2.10 Related Systems

Repair and home-service needs in Bangladesh are served today in three broad ways:

- **Local repair shops and freelance technicians**, found through word of mouth or social media. Prices are negotiated in person, there is no tracking and little recourse.
- **Brand service centres**, which offer reliable repair for their own products but cover only one brand and are concentrated in large cities.
- **General on-demand service marketplaces** such as Sheba.xyz in Bangladesh [15] and Urban Company in South Asia [16], which let customers book many kinds of household services from listed providers.

International platforms such as Thumbtack [17] popularised the model in which customers describe a job and several professionals respond with quotes. Sarabo adopts this offer model but specialises it for electronics repair, where the true price is usually unknown until the device has been opened. Table 2.1 compares the approaches on the features most relevant to this project.

Table 2.1: Comparison of approaches to electronics repair

Capability	**Local shop**	**General marketplace**	**Sarabo**
Online request with device and problem details	–	✓	✓
Price estimate before hand-over	–	Partial	✓
Several competing offers per job	–	Varies	✓
Separate inspection fee credited to final price	–	–	✓
Formal inspection and itemised quote	–	Varies	✓
Online payment with receipt	–	✓	✓
Step-by-step public tracking	–	Partial	✓
Technician matched by repair-specific expertise	–	Partial	✓
Bangla/Banglish AI request assistance	–	–	✓

Sarabo's contribution is not a new algorithm. Its engineering value lies in combining patterns from ticketing systems (a trackable case with an identifier and history), field-service systems (assignment, inspection, completion), marketplaces (offers, payment, commission, payouts) and evidence-driven workflows (private photos and completion records) into one coherent repair workflow with explicit state, money and authorization rules.

Chapter 3

Requirements Analysis

3.1 Stakeholders and Roles

The system has three signed-in roles and one anonymous audience. Their goals are summarised in Table 3.1.

Table 3.1: User roles

Role	Primary goal	Representative actions
Customer	Get a device repaired at a fair, known price	Create request (optionally with AI help), add photos, pick a pickup slot, compare offers, pay inspection fee, approve or reject quote, pay, track, confirm receipt, review or report the technician, claim warranty
Technician	Find suitable work and get paid	Apply with expertise and region, browse the jobs board, send offers, inspect, quote, revise, repair, complete, view wallet, withdraw via bKash
Admin	Keep the service healthy	Approve technicians, edit catalogue and prices, handle missed pickups, refunds, disputes and warranty reviews, process withdrawals, moderate reviews and reports, read reports
Visitor	Learn about the service	Browse services, service areas and open jobs, track a repair with a tracking code, ask the AI support chat

3.2 Functional Requirements

Table 3.2 lists the functional requirements. Requirements FR-21 to FR-32 were added in the second development stage, and FR-33 to FR-38 in the final phase.

Table 3.2: Functional requirements

ID	Requirement
FR-01	The system shall authenticate users (email/password and Google) and protect private routes.
FR-02	The system shall distinguish Customer, Technician and Admin capabilities on the server.
FR-03	A verified Customer shall be able to create a structured repair request for a catalogue service.
FR-04	The service catalogue shall provide active device and repair categories with BDT estimate ranges, inspection fee caps (never below the minimum fee) and required expertise level.
FR-05	Customers shall be able to attach up to three damage photos through a controlled private-storage workflow.
FR-06	Users shall be able to apply as Technicians with region, district and structured expertise; Admin shall approve or reject applications.
FR-07	The assigned Technician shall submit an inspection before a quote can be submitted.
FR-08	The Technician shall submit an itemised quote (labour, parts, additional charges).
FR-09	The Customer shall approve or reject the quote; after rejection the Technician may revise once more or close the request.
FR-10	Payment shall be possible only for an approved quote and never twice.
FR-11	The Technician shall record repair progress and completion with optional evidence photos.
FR-12	The Customer shall confirm receipt of the repaired device.
FR-13	The system shall send role-appropriate in-app notifications for workflow events.
FR-14	A public tracking page shall show a sanitized timeline by tracking code, without personal or payment data.
FR-15	A 90/10 Technician/platform settlement shall be frozen when the repair is paid.
FR-16	Technicians shall request withdrawals up to their server-computed available balance; Admin shall mark each as paid (with bKash TrxID) or rejected exactly once.
FR-17	A customer shall be able to cancel a request before a technician is booked.
FR-18	Admin shall be able to edit the service catalogue; price changes shall create a new version without changing existing requests.
FR-19	Technicians shall be able to edit their expertise; Admin shall see technician availability.
FR-20	Customers shall be able to download a PDF receipt for every payment.
FR-21	A request shall be accepted only if at least one approved Technician in the customer's region can perform it; refused demand shall be counted.
FR-22	Customers shall choose a two-hour pickup slot (Saturday–Thursday, up to 7 days ahead) subject to per-region capacity, and may reschedule before pickup.
FR-23	Open requests shall appear on a public jobs board; matching Technicians shall be notified and may send offers (estimate range, inspection fee, note).
FR-24	The Customer shall compare offers and choose one, or reject one (the job is reposted and the Technician may send one revised offer).

ID	Requirement
FR-25	Choosing an offer with a fee shall open a Stripe payment; the confirmed payment shall book the Technician atomically.
FR-26	The inspection fee shall be credited toward the final quote, refunded when the visit fails or is cancelled in time, and split 90/10 when kept.
FR-27	Customers shall rate a completed repair (1–5 with optional comment) or file a private report; Admin shall moderate both.
FR-28	The system shall email request confirmations, pickup reminders and new-job alerts, and post in-app pickup reminders daily.
FR-29	Admin shall see an attention list (overdue and missed pickups, unmatchable requests, no offers in 24 hours, refunds waiting) and period reports.
FR-30	An AI helper shall turn a free-text description and/or damage photo into a draft request that the customer reviews.
FR-31	Technicians shall see a price guide from past offers and may ask AI to draft their offer note.
FR-32	An AI support chat shall answer visitor questions from live catalogue data, and Admin reports shall include an AI summary checked against the real figures.
FR-33	Offers shall be ranked by rating, reliability, price, district and reply time, with the best match and its reasons shown; the customer still chooses.
FR-34	A finished repair shall carry a 30-day warranty: free re-inspection by the same technician, free re-repair if covered, and an admin refund if it cannot be fixed.
FR-35	Every waiting state shall have a deadline, an automatic step or an admin alert (unpaid quote, not repairable, repair that cannot be finished, receipt not confirmed, device not returned, stalled work).
FR-36	The technician shall record the device's condition and accessories at pickup, and may report a customer who is not home without it counting as a no-show.
FR-37	Every repair shall have an inspection fee of at least 100 BDT (200 BDT when the estimate exceeds 1,000 BDT).
FR-38	Technicians shall edit their own name, phone, address and service area (the area only while no repair is active).

3.3 Non-Functional Requirements

3.4 Service Catalogue

The catalogue contains **45 services** across eight device categories (Table 3.4). Each service pairs a device category with a repair category and carries a BDT estimate range, an inspection-fee cap, a required expertise level (beginner, intermediate, advanced, expert) and an estimated duration. Prices are shown as estimates; the final amount is always the technician's written quote

Table 3.3: Non-functional requirements

Category	Requirement
Security	Identity, role, prices, fees, balances and refund decisions are derived or validated on the server. Private images are reachable only through short-lived signed URLs. Secrets never reach the browser.
Consistency	Steps that change several records (booking, approval, completion, settlement, refunds) run as database transactions.
Reliability	Repeated or concurrent actions return controlled conflicts instead of repeating money or state changes. Failed refunds stay pending and can be retried. Email or notification failures never fail the main action.
Privacy	Public pages (tracking, jobs board) expose no names, emails, addresses or payment data. AI providers receive no personal identifiers.
Cost control	Every AI feature has per-user and site-wide daily limits stored in the database, and is off unless explicitly enabled.
Usability	Each screen shows the current stage and the next action for that role; empty, loading and error states are truthful.
Accessibility	Keyboard access, visible focus, sufficient contrast, labelled controls, reduced-motion support, layouts from 320 px.
Performance	Cached server state on the client, code-split routes, direct-to-storage uploads, paginated lists.
Maintainability	Workflow rules, status presentation and query keys are centralised; CI runs lint, build and tests on every push.
Auditability	Tracking events, assignment history, quote history, payment rows, refund state, withdrawal processing and moderation history are kept.

after inspection.

Table 3.4: Service catalogue summary with sample services

Device	Services	Sample service	Estimate (BDT)
Smartphone	8	Screen Replacement	1,500–6,000
Laptop / Computer	8	Motherboard Repair	2,500–12,000
Television	5	Screen / Panel Repair	4,000–18,000
Refrigerator	6	Compressor / Refrigerant Service	3,500–15,000
Washing Machine	6	Control Board Repair	2,000–7,000
Air Conditioner	6	Gas Refill / Compressor Repair	3,000–20,000
Microwave Oven	4	Electrical / Power Repair	1,500–5,000
Other Electronics	2	General Repair	1,000–5,000
Total	**45**		

3.5 Use-Case View

Figure 3.1 shows the main interactions of each role with the platform.

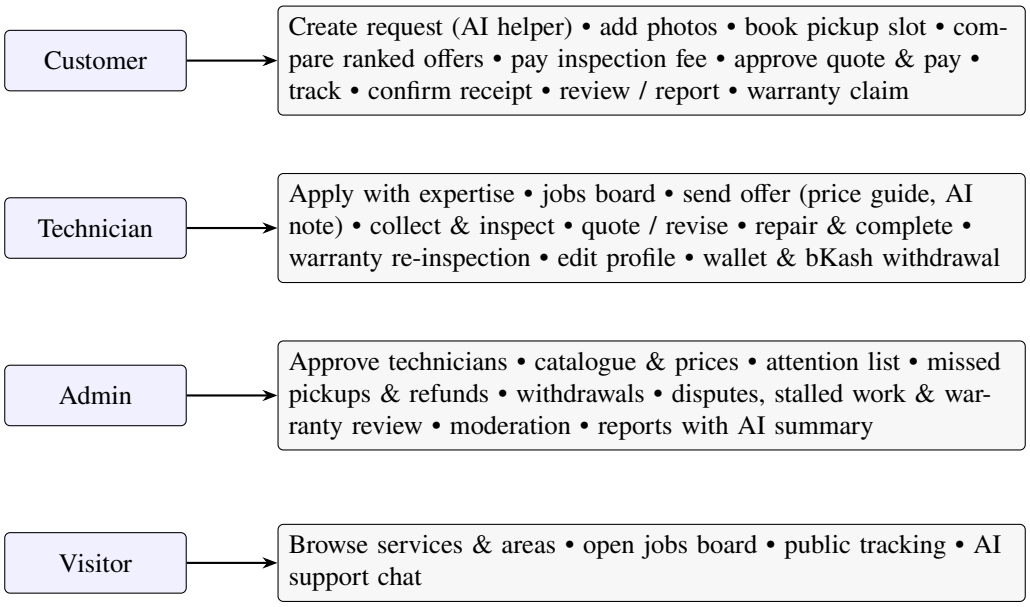


Figure 3.1: Role-oriented use-case view

Chapter 4

System Design and Architecture

4.1 High-Level Architecture

Sarabo is a browser client and an API server, with external services for identity, payment, storage, email and AI (Figure 4.1). Both run on Vercel; the API is a serverless function in Mumbai, next to the MongoDB Atlas cluster that holds all data, and Vercel also triggers a daily job.

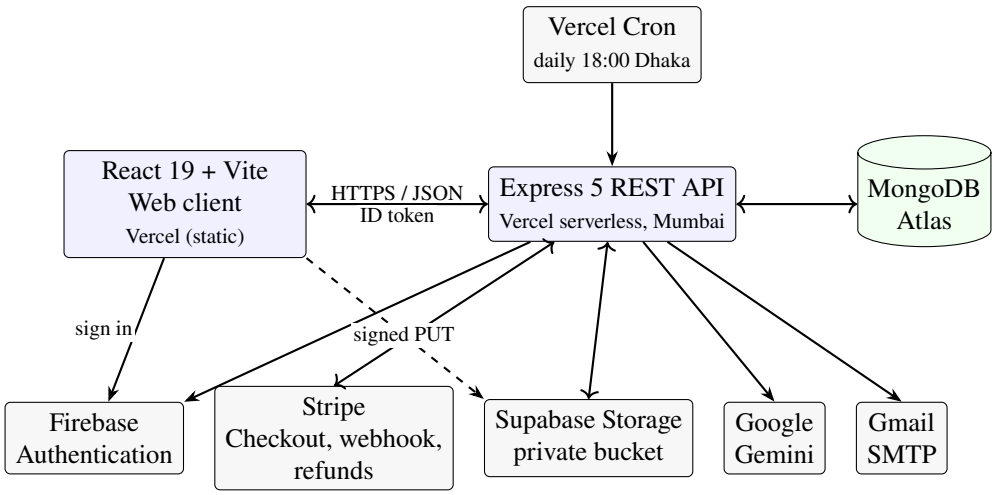


Figure 4.1: Sarabo high-level architecture

4.2 Client Architecture

The client (about 337 source files) is organised around five concerns:

1. **Routing and guards**: public pages (home, services, areas, jobs, tracking, about, become a technician), sign-in pages, and a dashboard split by `CustomerRoute`, `TechnicianRoute` and `AdminRoute`; the server re-checks every rule.

2. **Feature components**: request and AI helper, photos, pickup slots, offers, inspection, quote, payment, repair, warranty, feedback, wallet, notifications, support chat, administration.

3. **Server-state hooks**: TanStack Query with central query keys for caching, loading and error states, mutations and targeted invalidation.

4. **Presentation utilities**: the status-to-stage "service spine", currency formatting, eligibility helpers, per-action error messages.

5. **Design system**: shared tokens, light and dark themes, and Radix UI primitives.

4.3 Server Architecture

The server (about 191 JavaScript files with tests) is layered *routes* → *middleware* → *controllers* → *services* / *utilities* → *database*. Middleware adds a request ID, logs safely, verifies the Firebase token and waits for the database. Controllers own one domain's state rules; services hold shared logic (eligibility, booking, refunds, settlement, notifications, matching, AI, email); external providers sit behind adapters that tests replace with fakes.

The Stripe webhook route is registered before the JSON parser, since signature checks need the raw body. Routes stay registered while the database is briefly down, so requests fail with a controlled 503, not a misleading 404.

4.4 Data Collections

Table 4.1 lists the 21 MongoDB collections.

4.5 Repair Request Document

Requests carry `schemaVersion: 2`, a `product` snapshot (category, brand, model) and a copy of the chosen service with its price rule, so catalogue edits never change an existing request (Fig-

Table 4.1: MongoDB collections

Collection	Purpose
`users`	Accounts and the authoritative role.
`technicians`	Application, approval status, expertise, region/district, work status, bKash details.
`repair_requests`	The repair case with all lifecycle sub-documents.
`job_applications`	Technician offers on open requests and their states.
`service_definitions`	Versioned catalogue of services and prices.
`payments`	Stripe payment rows for inspection fees and repairs, including refund state.
`checkout_sessions`	Checkout idempotency and expiry.
`technician_withdrawals`	Withdrawal requests and admin processing.
`technician_reviews`	Customer ratings with moderation history.
`technician_reports`	Private customer reports with status and admin notes.
`notifications`	In-app notifications with de-duplication keys.
`tracking_events`	Sanitized public timeline.
`damage_upload_sessions`	Customer photo upload sessions.
`repair_evidence_sessions`	Technician completion-evidence sessions.
`deletion_cleanups`	Retry records for storage clean-up after deletion.
`turned_away_requests`	Hashed count of refused demand (180-day expiry).
`email_log`	Sent/failed email record (90-day expiry).
`support_rate_limits`	Shared AI rate-limit counters (auto-expiring).
`ai_report_summaries`	Last AI summary per report period.
`warranty_claims`	Warranty claims with their full decision and refund history.
`warranty_photo_uploads`	Short-lived upload sessions for warranty photos.

ure 4.2).

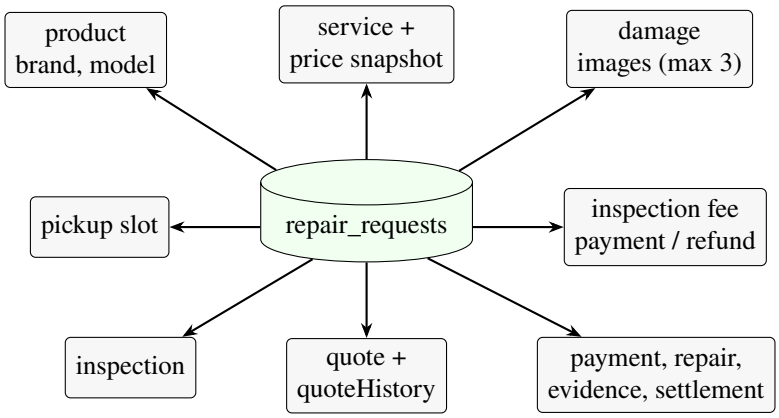


Figure 4.2: Conceptual structure of a V2 repair request

4.6 State Model

The status field `deliveryStatus` and several values come from the original parcel-delivery codebase; they are a frozen internal contract, and users see only repair terms (Table 4.2).

Table 4.2: Repair lifecycle states

Stored state	Meaning shown to users
`pending-pickup`	Waiting for a technician; open on the jobs board.
`assignment_pending`	Offered directly by an admin; technician to accept or reject.
`driver_assigned`	Technician booked.
`rider_arriving`	Technician on the way.
`parcel_picked_up`	Device collected.
`inspection_completed`	Inspection submitted.
`quote_submitted`	Final price waiting for the customer.
`quote_approved`	Customer accepted the price.
`quote_rejected`	Customer declined the price.
`payment_completed`	Customer paid.
`repair_in_progress`	Repair underway.
`repair_completed`	Repair finished; waiting for hand-over.
`parcel_delivered`	Customer confirmed receipt (terminal).
`cancelled`	Ended through an allowed path.

A job application (offer) has its own states: `pending`, `accepted`, `not_chosen`, `withdrawn`, `no_show` and `declined`. A settlement moves from *pending* to *available* (or to *reversed* after a refund), a warranty claim from *submitted* through re-inspection and decision to *resolved* or *refunded*, and a withdrawal from *requested* to *paid* or *rejected*.

4.7 The Service Spine

The interface folds the states into four stages customers understand (Figure 4.3), shown on the home page, the tracking page and every dashboard; receipt confirmation is a hand-over action after "Repaired", not a fifth stage.

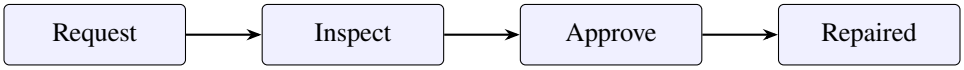


Figure 4.3: The four-stage service spine shown to users

4.8 API Design

The API has over 120 endpoints in resource families; Table 4.3 lists representative ones and Appendix A a fuller list.

Table 4.3: Representative API endpoints

Method	Route	Purpose
POST	`/repair-requests`	Create a request
GET	`/jobs`	Public open-jobs board
POST	`/repair-requests/:id/applications`	Technician sends an offer
POST	`/repair-requests/:id/applications/:aid/accept`	Customer chooses an offer
POST	`/repair-requests/:id/inspection`	Submit inspection
POST	`/repair-requests/:id/quote/decision`	Approve or reject quote
POST	`/repair-requests/:id/checkout-session`	Start repair payment
POST	`/stripe-webhook`	Signed payment events
POST	`/repair-requests/:id/repair/complete`	Complete repair
POST	`/repair-requests/:id/review`	Rate the technician
POST	`/ai/request-draft`	AI request helper
GET	`/admin/attention`	Items needing admin action
GET	`/public/trackings/:code`	Sanitized tracking

Chapter 5

Implementation

5.1 Technology Stack

Table 5.1: Implemented technology stack

Layer	Technology	Role in Sarabo
Frontend	React 19, Vite 7	Component-based single-page app and build tooling
Routing	React Router 7	Public, authenticated and role-specific routes
Server state	TanStack Query 5, TanStack Table	Caching, mutations, invalidation; admin data tables
UI	Tailwind CSS 4, Radix UI, Motion, Recharts, Leaflet	Design system, accessible primitives, animation, charts, service-area map
Forms	react-hook-form	Validated multi-step forms
Backend	Node.js, Express 5	REST API, middleware, controllers, services
Database	MongoDB 7 driver, MongoDB Atlas	Persistence and multi-document transactions
Identity	Firebase Authentication + Admin SDK	Sign-in (email, Google) and token verification
Payments	Stripe 20	Checkout, signed webhooks, refunds
Storage	Supabase Storage	Private images with signed URLs
AI	Google Gemini (`@google/genai`)	Support chat, request helper, offer note, report summary
Email	Nodemailer (Gmail)	Request, reminder and job-offer emails
Documents	pdfkit	PDF payment receipts
Testing	Custom Node test harness, Playwright	Server integration tests; browser tests
Delivery	GitHub Actions, Vercel	CI, serverless API (Mumbai region), cron, static hosting

5.2 Development Process

Sarabo was built in small, reviewable phases from 26 July to 4 October 2026, plus a final improvement phase from 11 to 16 October: 163 client and 120 server commits. Each phase followed one loop: define the rule, implement it on the server with tests, build the interface, run lint, build and tests, review. Larger features used branches merged by pull request once CI passed (Table 5.2).

Table 5.2: Development phases (from the git history)

Phase	Dates	Main work
1. Foundation and hardening	26–28 Jul	Imported the base codebase; secured authentication and role routing; secure Stripe checkout and webhook; validated status transitions; public sanitized tracking; transactional assignment, approval and completion.
2. Admin and notifications	29 Jul – 4 Aug	Admin request management; in-app notifications; production database isolation; health monitoring; double-booking prevention.
3. Repair domain (V2)	5–8 Aug	Service taxonomy and priced catalogue; technician expertise; V2 request schema; expertise-based matching; damage uploads; prices in BDT; inspection, quote, payment and repair workflows; safe deletion; verified email.
4. Rename to Sarabo	10–12 Aug	Migrated database, collections, fields, modules, API routes and UI from parcel to repair vocabulary; assignment accept/reject; receipt confirmation.
5. Money and UI redesign	13–19 Aug	90/10 settlement; wallet and withdrawals; storage moved to Supabase; complete "service spine" redesign; accessibility pass. *(Earlier report submitted 20 Aug.)*
6. AI support and feedback	18–24 Sep	Gemini support chat; reviews and private reports with moderation; product tour; second redesign; admin catalogue editor; CI on GitHub Actions.
7. Operations	24–25 Sep	Pickup scheduling; region rule; email updates; Playwright tests; admin reports and attention list; in-app reminders; refusing unmatchable requests.
8. Job portal and inspection fee	25 Sep – 2 Oct	Technician offers and customer choice; inspection-fee payment, credit, refunds and split; reject-and-repost; public jobs board; job notifications; PDF receipts; one-step approve-and-pay.
9. AI helpers and payouts	2–4 Oct	AI request helper (text and photo); price guide and AI offer note; AI report summary; bKash withdrawals; hosting configuration.

Phase	Dates	Main work
10. Final improvements	11–16 Oct	Map search, final payout on removal, faster API and AI, offer ranking, warranty, deadlines for every waiting state, pickup condition, minimum inspection fee, technician profile editing (Section 5.13).

5.3 Frontend Implementation

5.3.1 Role-Aware Routing

The dashboard sits behind an authenticated route with role guards. Customers get Overview, Create Request, My Requests and Payment History; Technicians get Overview, Find jobs, Assigned jobs, Completed jobs, Warranty claims and Wallet; Admins get Overview, Repair requests, Assign technicians, Technicians, Withdrawals, Warranty claims, Users, Services & prices, Reports, Technician reports and Technician reviews. All roles have Notifications and Profile; phones get a bottom tab bar. Deep links (e.g. `?category=smartphone`) survive sign-in. Routing is only a convenience: every protected endpoint re-checks authentication and authorization.

5.3.2 Server-State Hooks

Each feature has a query-key factory (e.g. `quoteKeys`, `walletKeys`) and read/mutate hooks. A mutation invalidates exactly the stale queries (approving a quote refreshes the quote, the request, the request list and payment eligibility), so all three roles stay consistent without page reloads.

5.3.3 Action-First Workspace

Each request page (e.g. Figure 7.12) shows the service spine, then a "next step" panel telling the signed-in role what to do (e.g. "Choose a technician" or "Approve and pay BDT 3,800"), then supporting details. Shared empty, loading, error, confirmation and form components keep behaviour consistent; a product tour introduces each dashboard, and a command menu supports keyboard navigation.

5.4 Backend Implementation

5.4.1 Controllers, Services and Adapters

Endpoints live in domain controllers (e.g. `jobPortalController`, `quoteController`, `paymentController`); shared logic lives in services such as `jobBooking`, `inspectionFeeRefunds`, `paymentEligibility`, `technicianMatchGate` and `notificationService`. External providers sit behind adapters (Supabase, Gemini, Stripe) that tests replace with fakes.

5.4.2 Database Transactions

Updates to several dependent records use MongoDB transactions. Booking after an inspection-fee payment, for instance, records the payment, sets the request to `driver_assigned`, marks the technician busy, accepts the chosen offer and marks the others `not_chosen` in one transaction (`services/jobBooking.js`); if any step fails, nothing changes.

5.4.3 Notifications, Email and Scheduled Jobs

Events create in-app notifications through one service, with de-duplication keys so a retry never notifies twice. Emails (request received, pickup reminder, new job) go through Gmail after the main action; a failed email is logged in `email_log` and never fails the request. A daily Vercel Cron job, protected by a secret, sends pickup reminders and applies every deadline in Section 6.8.

5.5 Service Catalogue and Matching

Admins edit services and prices from the dashboard; a price change increments `pricingRule.version`, while existing requests keep their price snapshot. Every service has an inspection fee of at least 100 BDT (200 BDT when its estimate tops out above 1,000 BDT). One rule, `minimumInspectionFee`, applies to requests, offers, the editor and the public price list, and a one-time migration raised the 16 live services that had no fee. On request creation `technicianMatchGate` checks that an approved technician in the region has the required device, repair and level; if not, the API answers `409 NO_TECHNICIAN_AVAILABLE` and the refused demand is counted once per customer, service, region and day (customer hashed), showing admins

where to recruit.

5.6 Pickup Scheduling

Customers pick one of four two-hour windows (10:00–18:00 Bangladesh time), Saturday to Thursday, one hour to seven days ahead. Each region takes a configurable number of pickups per slot (default three; a full slot returns `409 PICKUP_SLOT_FULL`). The window can change until collection, and a moved pickup is reminded again.

5.7 Quotation Model

A quote has whole-taka labour, parts and additional charges, a total, currency, version and status. A rejected quote moves to `quoteHistory` before the live quote is cleared, so it stays auditable and can never be paid by mistake.

Listing 5.1: Quote total and amount still to pay

```
repairSubtotal = laborAmount + partsAmount + additionalCharges
quote.totalAmount = repairSubtotal (currency "BDT")
amountDue = max(quote.totalAmount - inspectionFeePaid, 0)
```

5.8 Payment Implementation

Every Stripe Checkout session is created from stored data (the approved quote or the chosen offer's fee), never from a browser amount, and its metadata says what it pays for (`repair` or `inspection_fee`). The signed webhook and, as a fallback, the success page call the same idempotent processor, so a payment is recorded exactly once. One server function decides whether a request is paid, used by eligibility, admin filters and badges alike. Each payment has a PDF receipt (Stripe ID, Dhaka-time date, device, repair, technician, amount, refund status).

5.9 Technician Settlement and Wallet

When a repair is paid, a settlement snapshot is frozen from the approved quote:

Listing 5.2: Settlement rule (whole taka)

```
commission = round(repairSubtotal * 10 / 100) # Sarabo
receivable = repairSubtotal - commission # Technician
# invariant: commission + receivable == repairSubtotal
```

The settlement stays *pending* until the customer confirms receipt, then becomes *available*. Balances are never stored; they are recomputed from settlements and withdrawals, so they cannot drift. A technician withdraws up to the available balance to a bKash number; an admin sends it by bKash Send Money and records the TrxID. A unique partial index allows one open withdrawal per technician.

5.10 Damage and Evidence Images

Images go straight from the browser to a private Supabase bucket via a signed upload URL (JPEG, PNG or WebP, up to 8 MB, three per request). The server checks the stored object's real type and size before attaching it, and gives five-minute signed read URLs only to the owner, an admin or the assigned technician. Storage clean-up after deletion is retried until it succeeds.

5.11 Public Tracking and Jobs Board

Each request gets a tracking code `SRB-<random>` with 128 bits of randomness; `GET /public/trackings/:code` (exact codes, rate limited) returns only the status timeline. The jobs board (`GET /jobs`, 60 requests per minute per address) shows device, repair, area, pickup day, usual price and offer count, never the problem text, photos, tracking code or personal details.

5.12 Deployment and Continuous Integration

The API is a Vercel serverless function (Node.js 24) at `https://sarabo-server.vercel.app` and the client a static Vercel site at `https://sarabo-client-six.vercel.app`; both deploy from `main`. The API runs in Vercel's Mumbai region (`bom1`), next to the MongoDB Atlas cluster, which cut the first request after idle from 10.7 s to 2.9 s and a normal one from 0.75 s to about 0.2 s. Production has its own database, and the server will not start there without its secrets

and an `https` client origin. GitHub Actions runs on every push: the server suites on a throwaway MongoDB replica set (never with real credentials), and client lint, build and browser tests. Pushing to `main` happens only after all checks pass.

5.13 Final Improvement Phase

After feature completion on 4 October, a final phase (11–16 October 2026) prepared the project for submission, starting with a review of both codebases and a fresh run of every check (Chapter 10). Each change is recorded in the project work log with its problem, files and verification, and summarised in Table 5.3.

Table 5.3: Final improvement phase log

#	Date	Change and verification
0	11 Oct 2026	Codebase and documentation review; re-ran server suites against an isolated in-memory MongoDB replica set, client lint, production build and Playwright browser tests.
1	11 Oct 2026	Service-area map: OpenStreetMap coordinates for 226 of the 236 covered sub-areas, each kept only if it lies inside its own district. Searching a sub-area (e.g. Ambarkhana) now zooms to that spot instead of the district centre. Verified with two new browser tests on desktop and mobile.
2	11 Oct 2026	Final payout on technician removal: removing a technician queues their whole available balance as a bKash withdrawal in the same transaction (Section 6.10). Verified with 15 new server checks.
3	11 Oct 2026	Homepage hero: the photo moved to the right side (top band on phones) so the headline no longer sits over it. Checked with screenshots at three screen widths.
4	11 Oct 2026	Final verification of the committed code and CI on both repositories (Section 10.2).
5	14 Oct 2026	Slow first load: the API was moved to Vercel's Mumbai region, next to the database (10.7 s → 2.9 s cold, 0.75 s → 0.2 s warm, measured live).
6	14 Oct 2026	AI support timeouts: faster Gemini model with an automatic fallback model; live answers now take 1.4–1.9 s. 4 new tests.
7	14 Oct 2026	Smarter matching: offers ranked by rating, reliability, price, district and reply time, with a "Best match" badge and reasons (Section 6.4). 22 new tests.
8	14 Oct 2026	30-day repair warranty with free re-inspection, re-repair, admin review and refunds (Section 6.9). 45 new tests.
9	14 Oct 2026	Whole-workflow review, then fixes so no request can get stuck: unpaid quotes, not-repairable devices, repairs that cannot be finished, unconfirmed receipt, warranty deadlines (Section 6.8). 42 new tests.
10	14 Oct 2026	Remaining gaps: late payment after cancellation refunded, device return after cancellation, stalled-work alerts, pickup condition, customer not home. 30 new tests.
11	14 Oct 2026	Photos upload only when the customer presses "Upload"; technicians edit their own profile; minimum inspection fee for every repair, applied to old offers and migrated on production. 22 new tests.
12	14 Oct 2026	All of the above pushed and deployed; full verification (Section 10.2) and screenshots of the running system (Chapter 7).

#	Date	Change and verification
13	16 Oct 2026	Stripe's repair-payment page named the item "Repair request: undefined"; it now names the device ("Repair: Samsung Galaxy A54"). 1 new test; pushed and deployed.

Chapter 6

Core Workflows and Business Rules

6.1 End-to-End Repair Workflow

Figure 6.1 shows the main path of a repair, from request to settlement.

6.2 Request Creation

The customer picks a device and repair from the live catalogue (or lets the AI helper propose them), adds brand, model and a description, optionally up to three photos (sent only on "Upload"), and a pickup window; the estimate shows the catalogue price range. The form checks `GET /service-availability`, and the server checks again on submission: if no approved technician in the region can do the repair at the required level, the request is refused with an explanation. Accepted requests get a tracking code and a confirmation email (Figures 7.4 and 7.5).

6.3 Job Portal and Offers

A request waiting for a technician is *open*: it appears on the public jobs board as a safe summary, and each matching approved technician gets one `job_posted` notification (also when they start matching later, after approval or a new skill). Technicians see the district, pickup time, problem and photos, never the customer's name, email or street address.

An *offer* has an estimate range, an inspection fee (between the minimum fee and the catalogue fee) and a note. The customer chooses one, or rejects one with a reason (price, fee or other); the job

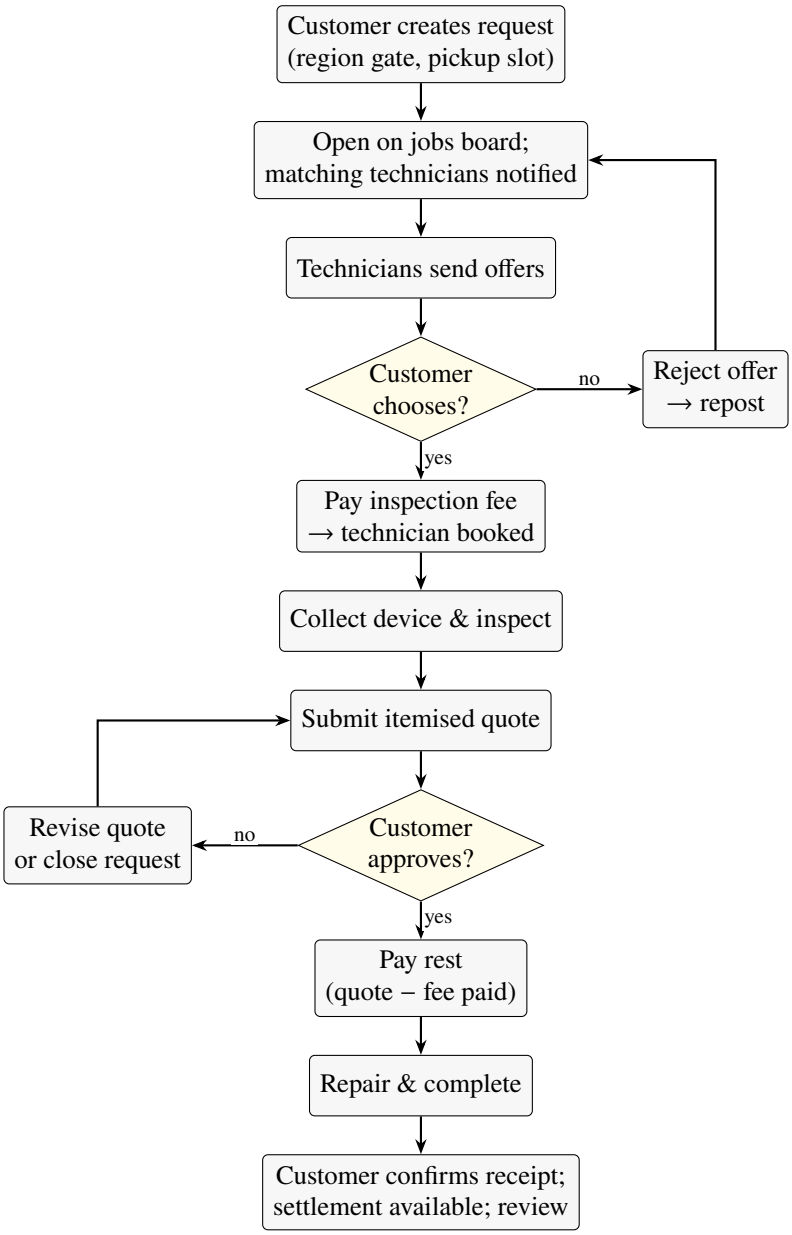


Figure 6.1: End-to-end repair workflow

is then *reposted*, the rejected technician may send one revised offer, and other matching technicians are notified. A job with no waiting offer 24 hours after its last posting goes on the admin attention list. Admins can also invite a technician to apply, or assign one directly as an override (Figures 7.6 and 7.7).

6.4 Offer Ranking (Smarter Matching)

Offers are ranked, not listed in arrival order. A pure scoring module (`services/offerRanking.js`) combines five factors from data Sarabo already stores, so no paid service, map API or AI is needed (Table 6.1); a technician with no history scores neutral on that factor. With two or more offers the top one gets a "Best match" badge, and reasons appear only when true (e.g. "Rated 4.8 ★ from 12 reviews", "In your district"). The customer can also sort by price or date, and always chooses; nothing is booked automatically.

Table 6.1: Offer ranking factors

Factor	Weight	How it is measured
Rating	30%	Average review, smoothed toward 3 ★ so one 5 ★ review does not beat forty 4.8 ★ reviews
Reliability	25%	Repairs completed compared with missed pickups
Price	20%	The offer's estimate compared with the other offers on the same job
District	15%	Same district as the customer
Reply time	10%	Median minutes from the job being posted to the technician's offer (last 20 offers, at least 3)

6.5 Inspection Fee and Booking

Choosing an offer opens a Stripe Checkout for exactly its fee. When Stripe confirms the payment, one transaction records it and books the technician; if booking is no longer possible (technician busy, request cancelled, another offer chosen), the payment is refunded automatically and the customer chooses again. The fee is stored apart from the repair payment and credited toward the final price (Figures 7.8 and 7.12). Table 6.2 lists what happens to it when the repair does not go ahead.

Refunds take two steps: the transaction that causes the refund marks the fee and its payment row *refund pending*; after it commits, Stripe is called with the idempotency key

Table 6.2: Inspection fee outcomes

Situation	Fee
Repair approved and paid	Credited toward the final price
Technician does not come (admin records a missed pickup)	Full refund; other offers can be chosen
Customer cancels 2 hours or more before the pickup window	Full refund
Customer cancels less than 2 hours before	Kept: 90% technician, 10% Sarabo
Customer declines the final price and the request ends	Kept: 90% technician, 10% Sarabo
Booking fails after payment	Automatic full refund

`inspection-refund-<session id>`, so at most one refund is made. A failed refund stays pending on the admin home page and can be retried with one click; it is never silently lost.

6.6 Inspection, Quotation and Revision

Only the booked technician can submit the inspection (findings and detected issues) after collecting the device, and a quote only from `inspection_completed`. The customer sees the quote with the fee already paid and the amount due, and the button "Approve and pay BDT *X*" leads straight to payment.

After a rejected quote the technician may revise or close the request (Figure 6.2); revising archives the old quote and repeats the submit-and-decide cycle. With no action for three days the daily job closes the request, records who closed it (`technician`, `customer` or `auto`), and the inspection fee is kept and split.

Each quote round has its own notification identity (`quoteHistory.length + 1`, derived on the server), so a retried submission is harmless while a real revision notifies again (Appendix E).

6.7 Payment, Repair and Receipt

Only an approved, unpaid quote can be paid. The technician then starts the repair, posts progress updates and completes it, optionally with evidence photos; completion frees the technician and asks the customer to confirm receipt. Only that confirmation ends the request and makes the earning available, so no technician can withdraw money for a device the customer has not received.

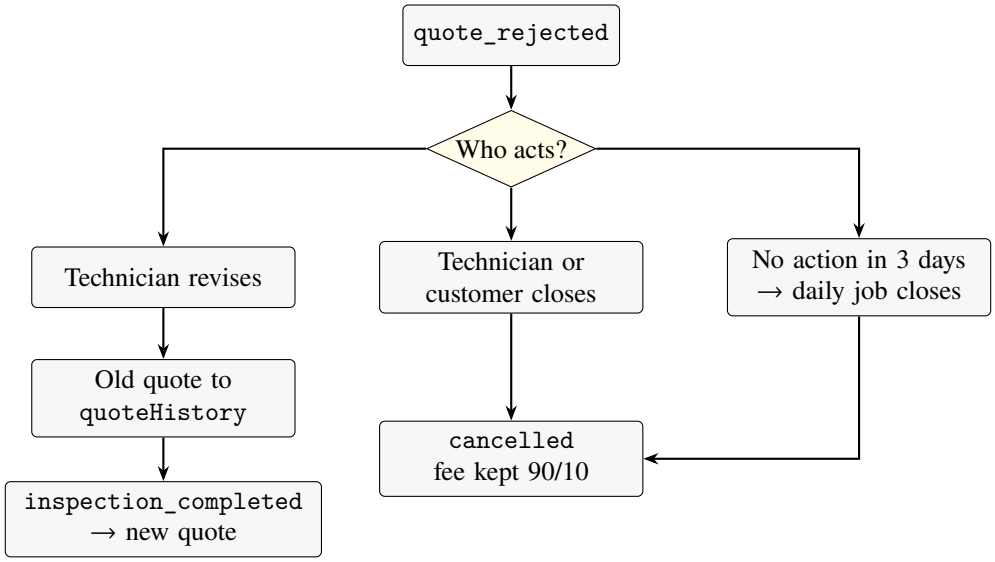


Figure 6.2: Rejected-quote recovery

6.8 No Request Can Get Stuck

A review before the final release looked for situations in which a request could wait forever. Each now has a way out, an automatic step after a deadline, or an admin alert (Table 6.3). A daily job applies the deadlines, and one rule set (`services/stalledWork.js`) lists work that has not moved for three days (fourteen for a running repair) on the admin overview.

Table 6.3: Situations that used to block a request, and how they end now

Situation	What happens now
Approved quote never paid	Customer can cancel (“I don’t want the repair”); otherwise closed automatically after 3 days. The inspection fee is kept 90/10 and the technician is freed.
Device not repairable or not worth it	Technician or customer closes the request; same fee outcome.
Paid repair cannot be finished	Technician reports why; an admin cancels and refunds the whole amount, and the technician’s settlement is reversed.
Customer never confirms receipt	Technician marks the device returned; receipt is confirmed automatically after 3 days unless the customer says “I haven’t received it”, which goes to an admin.
Device must go back after a cancellation	The return is recorded, confirmed by the customer or automatically after 3 days; disputes go to an admin.
Customer not at home at pickup	Technician reports it; the customer is asked for a new time. It is not counted as a technician no-show.

Situation	What happens now
Payment arrives after cancellation	Recorded and refunded at once; a failed refund stays on the admin refund list.
Work not moving (no inspection, quote, answer or repair)	Shown on the admin overview with one alert per request and stage.

Pickup condition. At collection the technician records the device's condition and accessories (charger, case, SIM). The customer sees the record and can dispute it, and an admin then reviews it; this protects both sides later (Figure 7.9).

6.9 Repair Warranty

Every finished repair is covered for 30 days from confirmed receipt: the same fault coming back or a fitted part failing, but not new damage, water damage or drops (Figure 6.3).

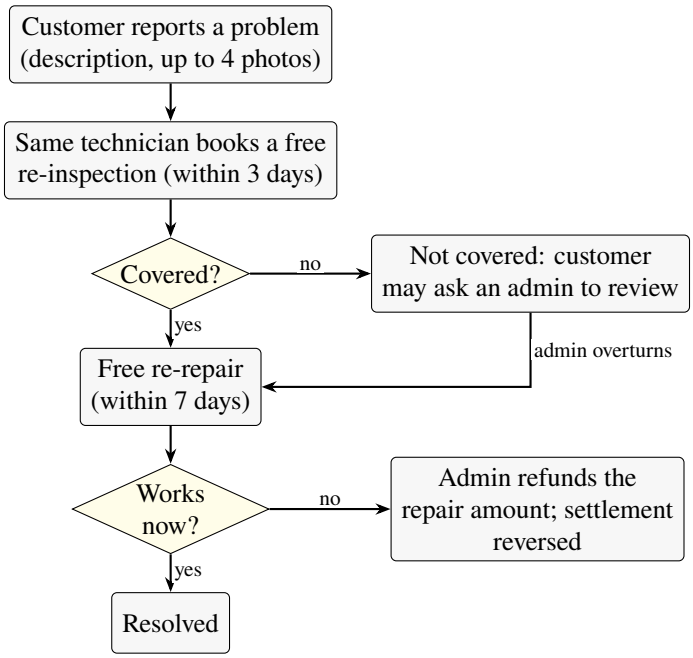


Figure 6.3: Warranty claim lifecycle

A customer may have one open claim at a time and two per repair. Each step is a guarded database update, so none can be skipped and two people acting at once cannot both succeed. A missed deadline marks the claim overdue and lets admins take it over, and a technician with open

claims cannot be removed. A refund covers the whole quote (repair payment plus credited fee) through Stripe, with one idempotency key per part; the settlement becomes *reversed*, and money already withdrawn is shown as owed and deducted from future earnings (Figures 7.16 and 7.17).

6.10 Settlement and Withdrawal Lifecycle

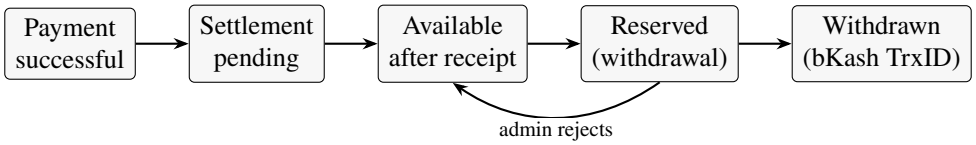


Figure 6.4: Technician settlement and withdrawal lifecycle

The wallet is computed, not stored:

$$\text{Pending} = \sum \text{paid settlements not yet receipt-confirmed} \tag{6.1}$$

$$\text{Gross available} = \sum \text{receipt-confirmed settlements} + \sum \text{kept inspection-fee shares} \tag{6.2}$$

$$\text{Reserved} = \sum \text{open withdrawal requests} \tag{6.3}$$

$$\text{Withdrawn} = \sum \text{paid withdrawals} \tag{6.4}$$

$$\text{Available} = \text{Gross available} - \text{Reserved} - \text{Withdrawn} \tag{6.5}$$

Because rejection simply removes a reservation, no compensating balance write is ever needed.

Final payout when a technician is removed. Removing an approved technician queues their whole available balance as a bKash withdrawal in the same transaction, so both happen or neither does. The number comes, in order, from the admin's entry, an already-open request (topped up, not duplicated), the last withdrawal, or the application phone; with money owed and no valid number, the removal is refused. Pending money is not included. The payout is tagged "final" and sent and recorded like any withdrawal, by a manual bKash Send Money, since Sarabo has no bKash API.

6.11 Cancellation Rules

Before booking a customer can cancel freely and later "Request again" with the details pre-filled. After booking with a paid fee, `GET /repair-requests/:id/booking-cancellation` shows in advance whether cancelling now refunds or keeps the fee; cancelling is allowed only until the device is collected.

6.12 Technician Profile Editing

Technicians edit their own name, phone, address and service area (region and district together), but not email, national ID, approval status or availability. The area is locked during an active repair, since its pickup was booked there; after a change the technician is told about open jobs in the new area.

6.13 Service-Area Map

The Service Areas page draws every district on an OpenStreetMap map (Leaflet) from the same `serviceAreas.json` that drives the request form. Sub-area coordinates were looked up once, offline, and kept only if inside their district (226 of 236). Search lists sub-areas before districts, so "Ambarkhana" opens at street level, not the city centre; a sub-area without a point falls back to its district's pin, and nothing is geocoded in the browser.

6.14 Feedback and Moderation

After a receipt-confirmed repair the customer may leave one review: 1–5 stars and an optional plain-text comment of up to 1,000 characters. Admins can hide or restore a review with a reason, and every change is kept in a moderation history. A customer can also file a private report about a technician (e.g. unprofessional behaviour, device damage, a payment issue); reports go only to admins, move through *open*, *under review*, *resolved* or *dismissed*, and are never deleted with the request. Technicians see their rating and visible reviews, never reports or reviewer identities.

6.15 Notifications and Reminders

Forty-seven notification types, defined in one module (`utils/notificationEvents.js`) and each with a de-duplication key, cover offers, booking, refunds, inspection, quotes, payment, repair, returns and disputes, warranty, reviews and withdrawals. The daily job sends one “pickup tomorrow” reminder per slot, in the app and by email when enabled.

6.16 Damage Image Upload

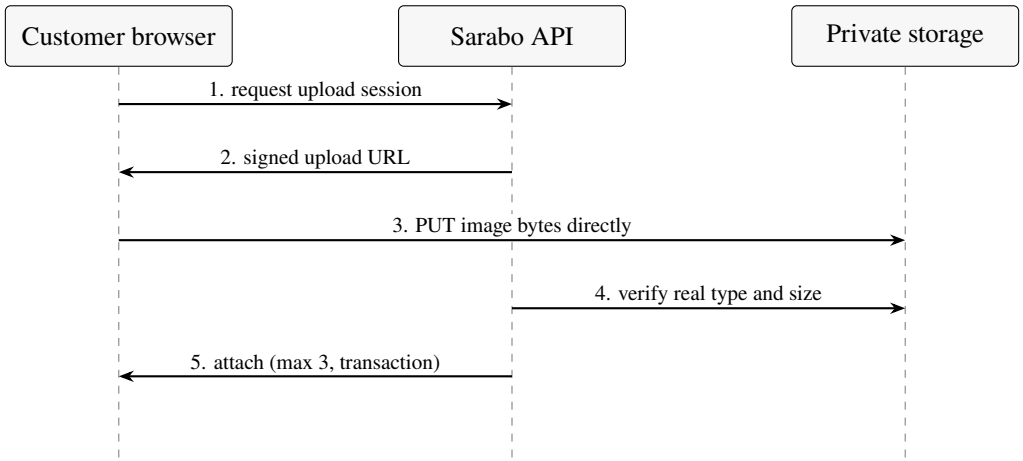


Figure 6.5: Damage-image upload workflow

Chapter 7

System Walkthrough

The screenshots below were taken on 14 October 2026 from the final code, with real Firebase sign-in, real Gemini answers and Stripe in test mode. From Section 7.2 they follow *one* repair, the cracked Samsung Galaxy A54 used throughout this report, through every state in Figure 6.1. Each picture shows only the part of the screen that matters; personal email addresses are blurred.

7.1 Public Website

Anyone can browse services, prices, service areas, open jobs and tracking without an account.

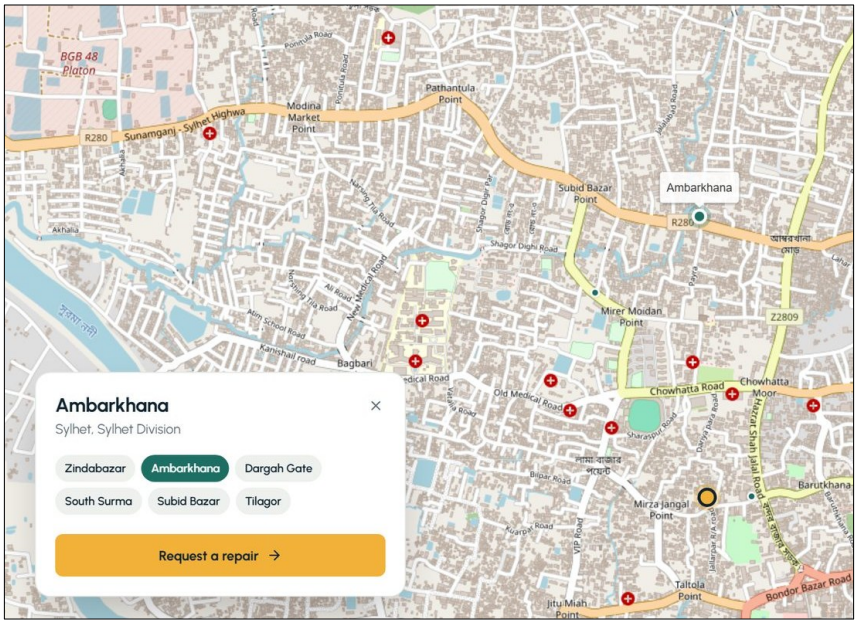


Figure 7.1: Service areas: searching “Ambarkhana” zooms the map to that part of Sylhet (Section 6.13).

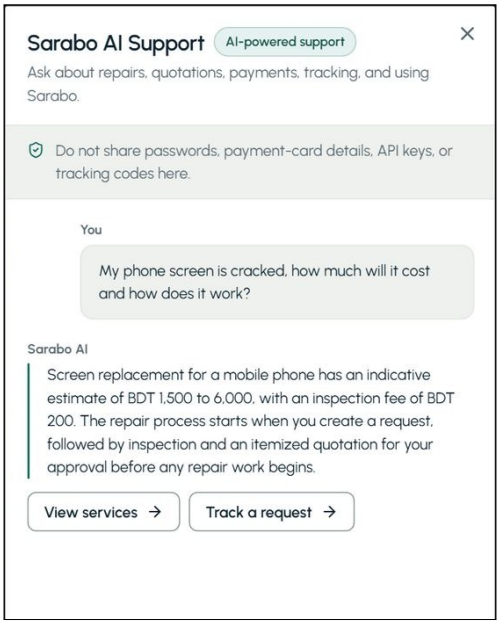


Figure 7.2: AI support chat: the price and the 200 BDT fee come from the live catalogue (Chapter 8).

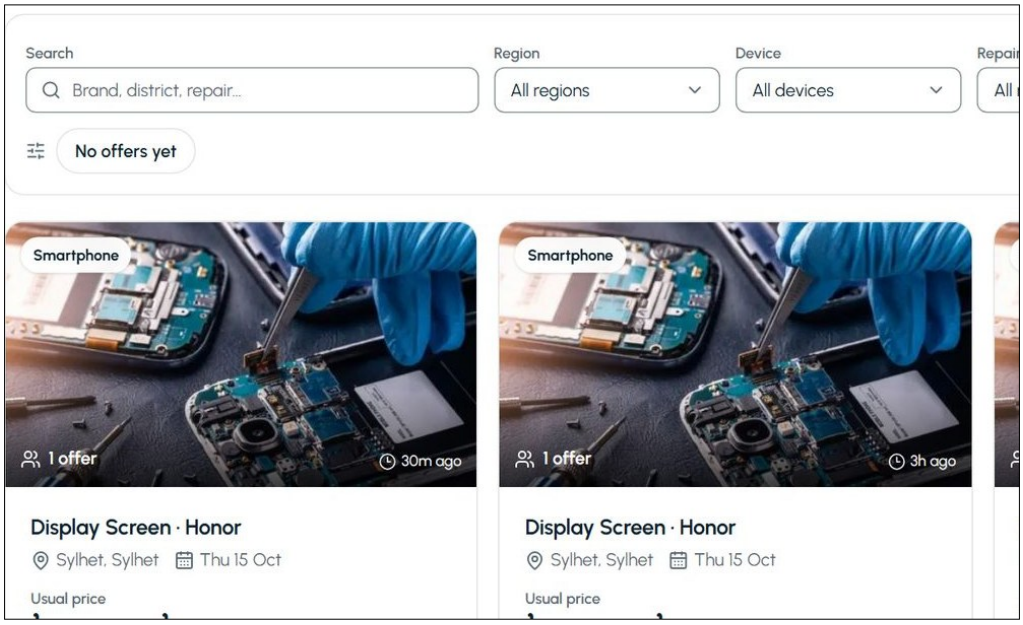


Figure 7.3: Public jobs board: a safe summary of each open job, with no customer details.

7.2 Request and Offer

Rahim describes the problem in Banglish and books a pickup; a local technician sends an offer.

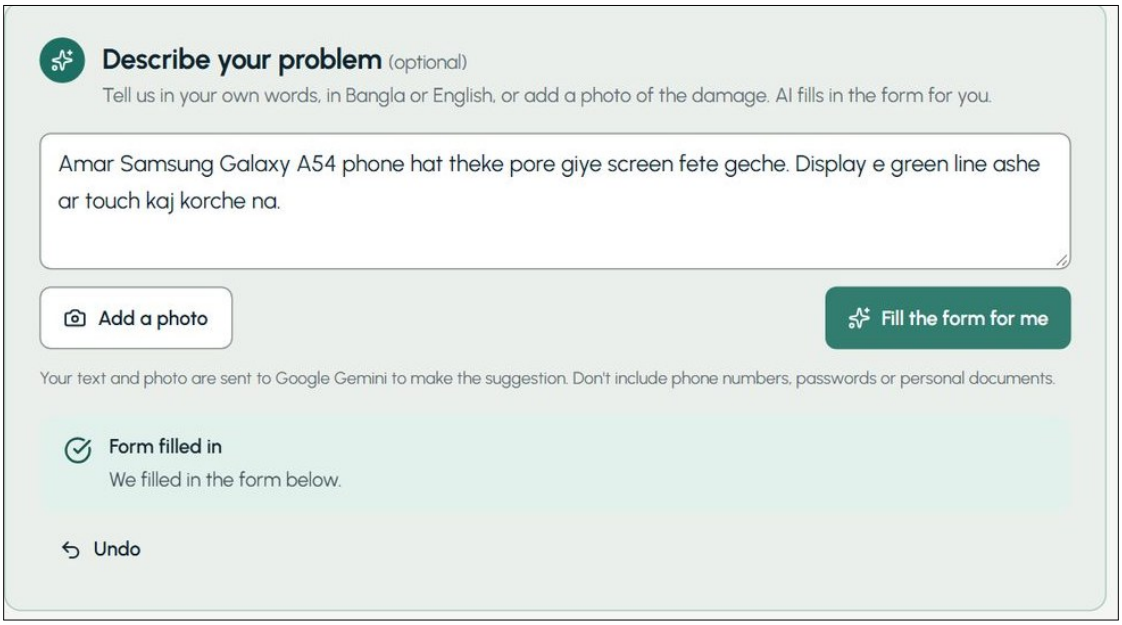


Figure 7.4: AI request helper: the Banglish text became the device, the catalogue service and an English description.

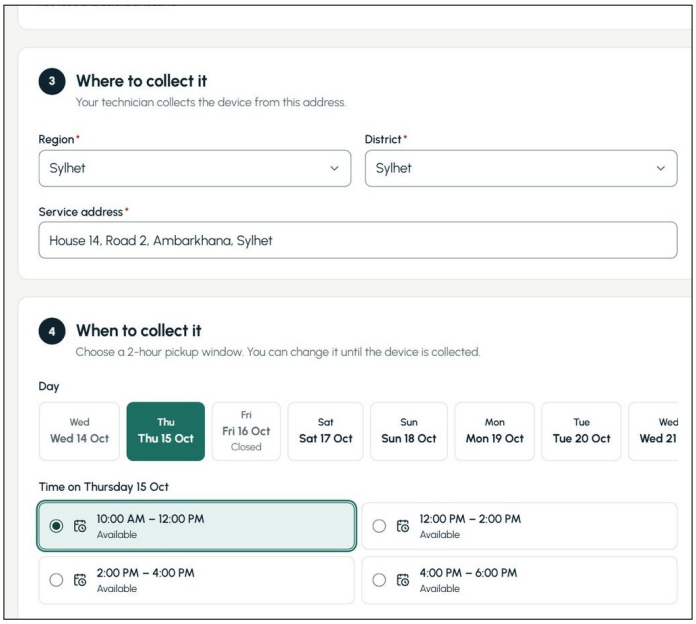


Figure 7.5: Address and a two-hour pickup window (Friday closed).

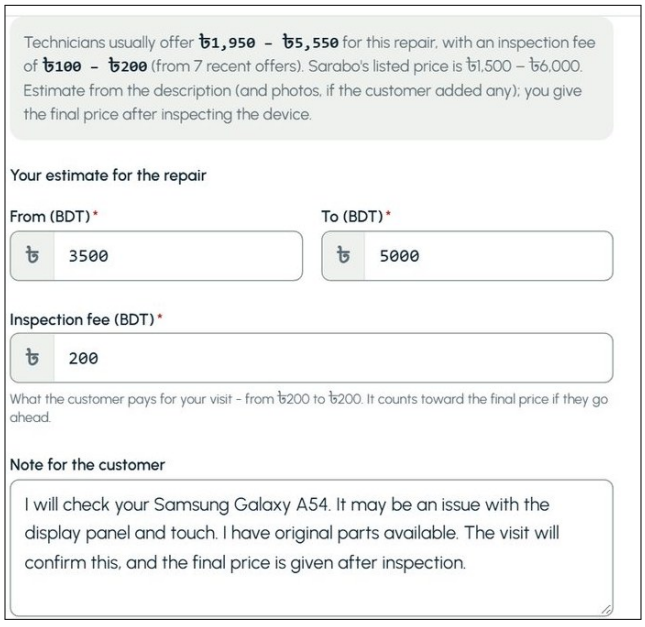


Figure 7.6: Technician's offer: price guide, minimum fee, and a note written by AI from a few words.

7.3 Choosing and Booking

He compares offers and pays the inspection fee to book the visit.

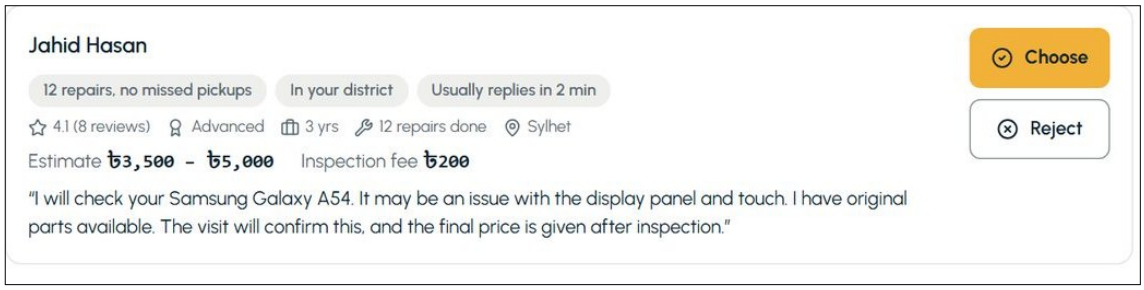


Figure 7.7: The offer as the customer sees it, with its ranking reasons (Section 6.4).

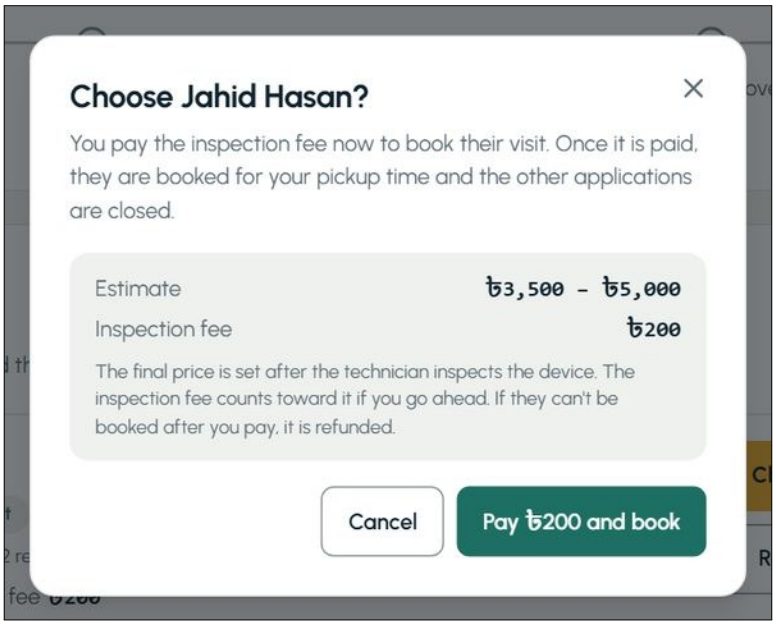


Figure 7.8: Choosing the offer: the inspection fee is paid by Stripe (test mode) to book the visit.

7.4 Pickup, Inspection and Quote

The technician records the condition, inspects the phone and sends an itemised quote.

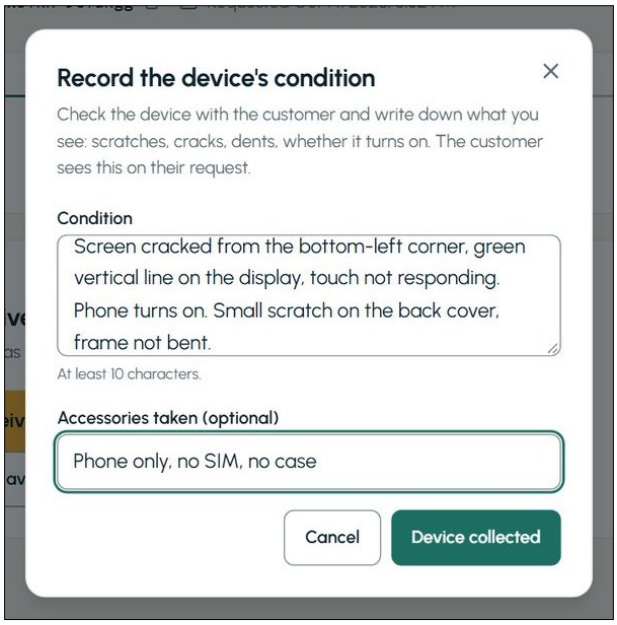


Figure 7.9: The device's condition and accessories recorded at pickup.

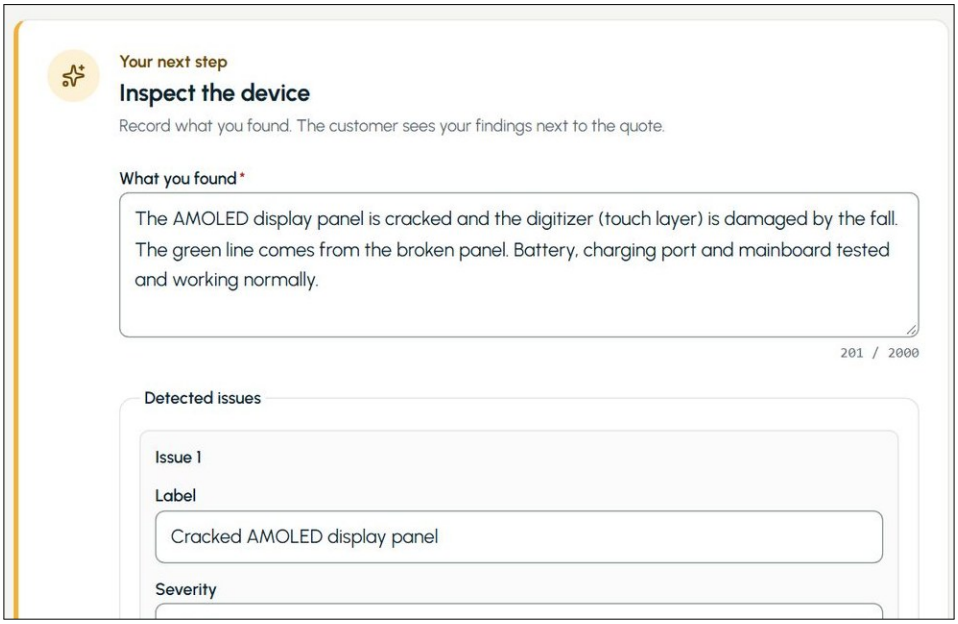


Figure 7.10: Inspection: findings, issues and a repairability decision.

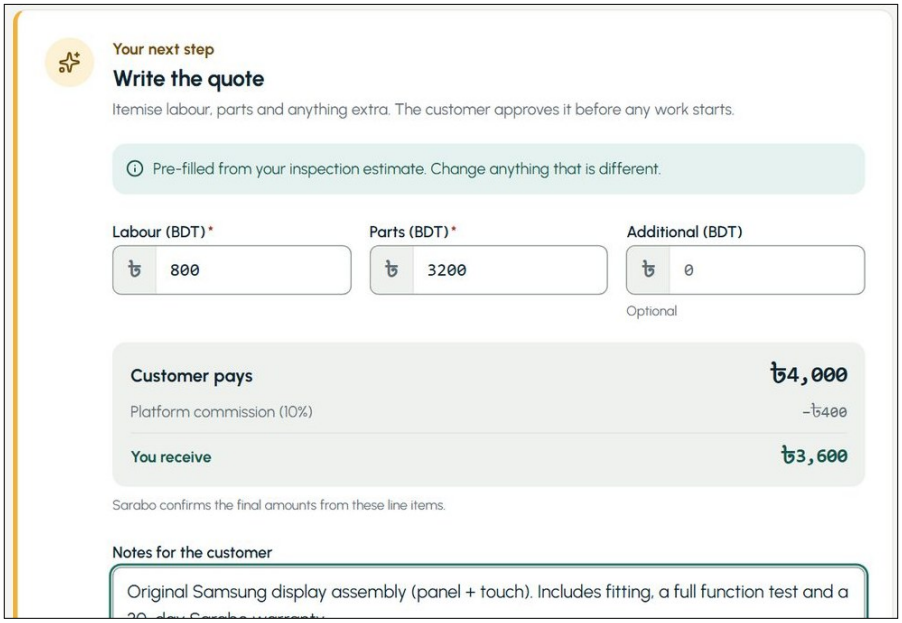


Figure 7.11: Itemised quote (800 + 3,200 = 4,000 BDT), with the 10% commission shown.

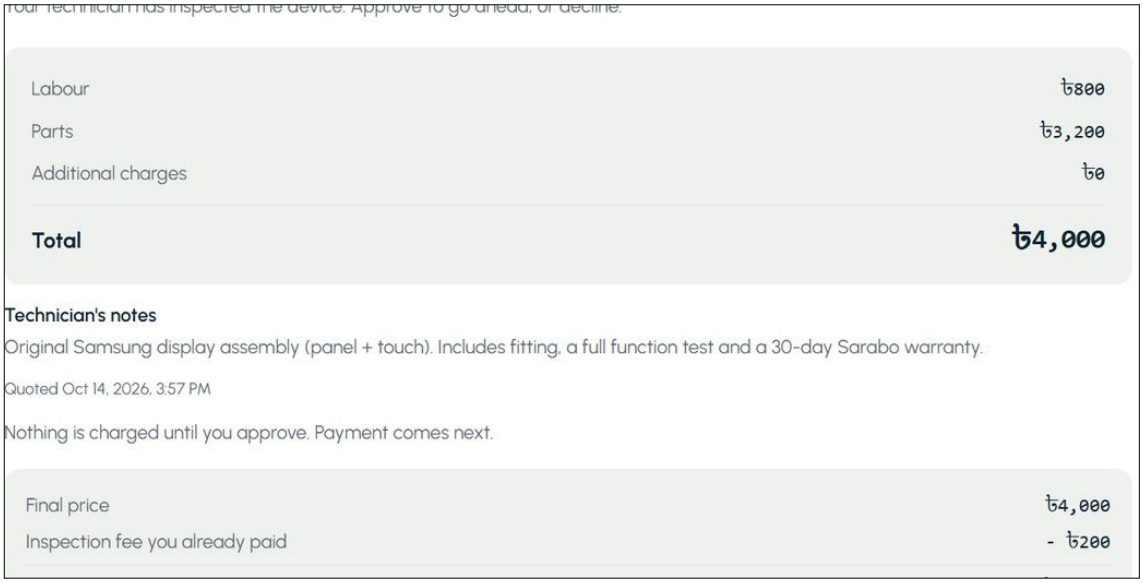


Figure 7.12: The customer's view of the quote: nothing is charged before approval.

7.5 Payment, Repair and Hand-Over

Rahim pays the rest; the technician repairs and returns the phone.

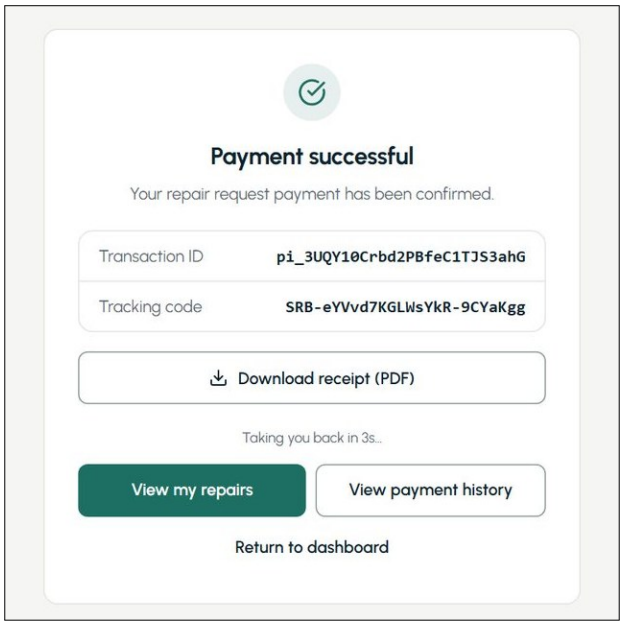


Figure 7.13: Payment successful (3,800 BDT after the 200 BDT fee), with a PDF receipt.

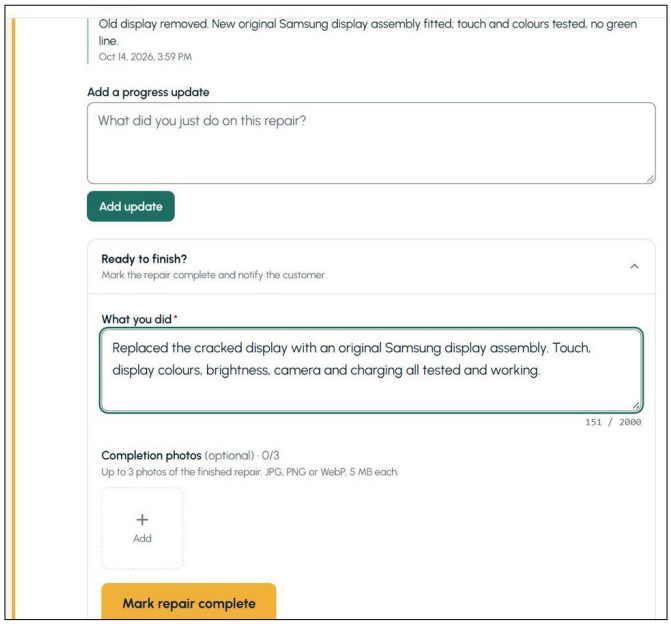


Figure 7.14: Completing the repair; “Can’t finish this repair” would send it to an admin.

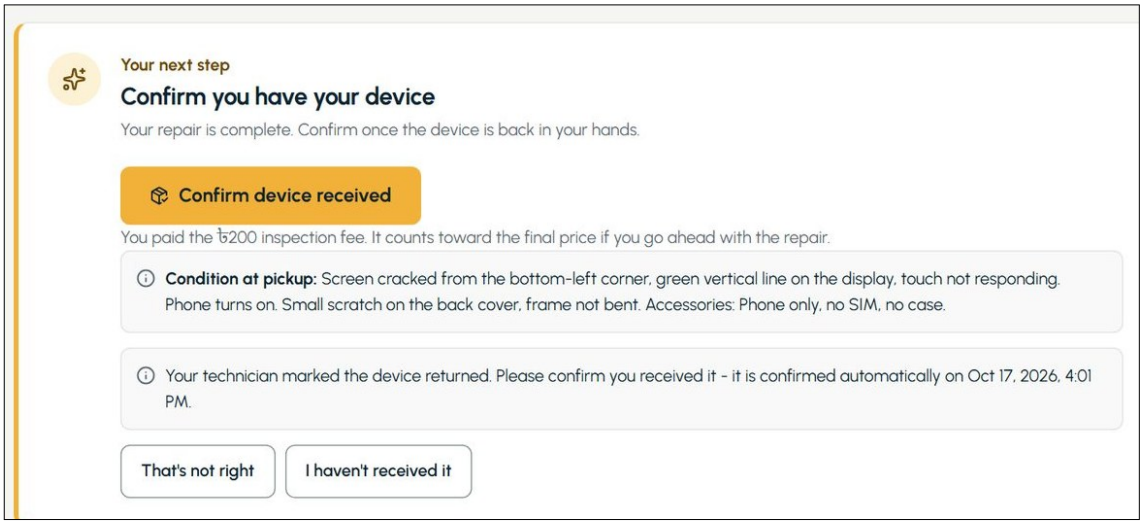


Figure 7.15: Confirm receipt, or "I haven't received it"; confirmed automatically after 3 days.

7.6 Warranty

Ten days later the touch fails again, and the warranty covers it.

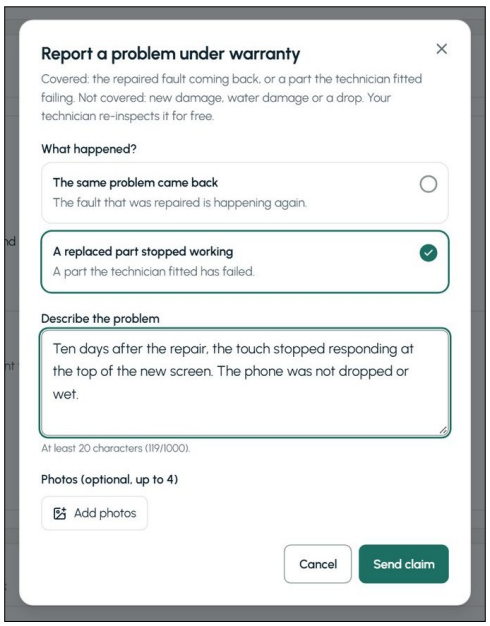


Figure 7.16: Reporting a problem under the 30-day warranty.

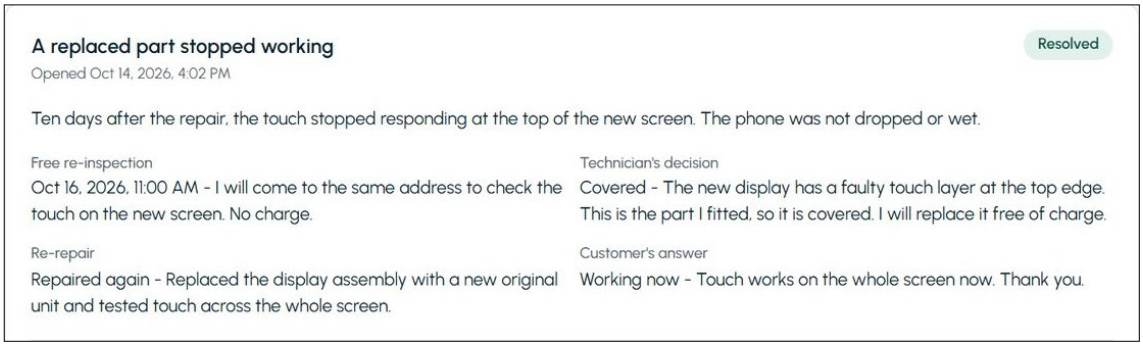


Figure 7.17: The resolved claim: free re-inspection, "Covered", re-repair and the customer's "Working now".

7.7 Technician and Admin

The technician withdraws the earning; the admin sees what needs attention and records the payout.

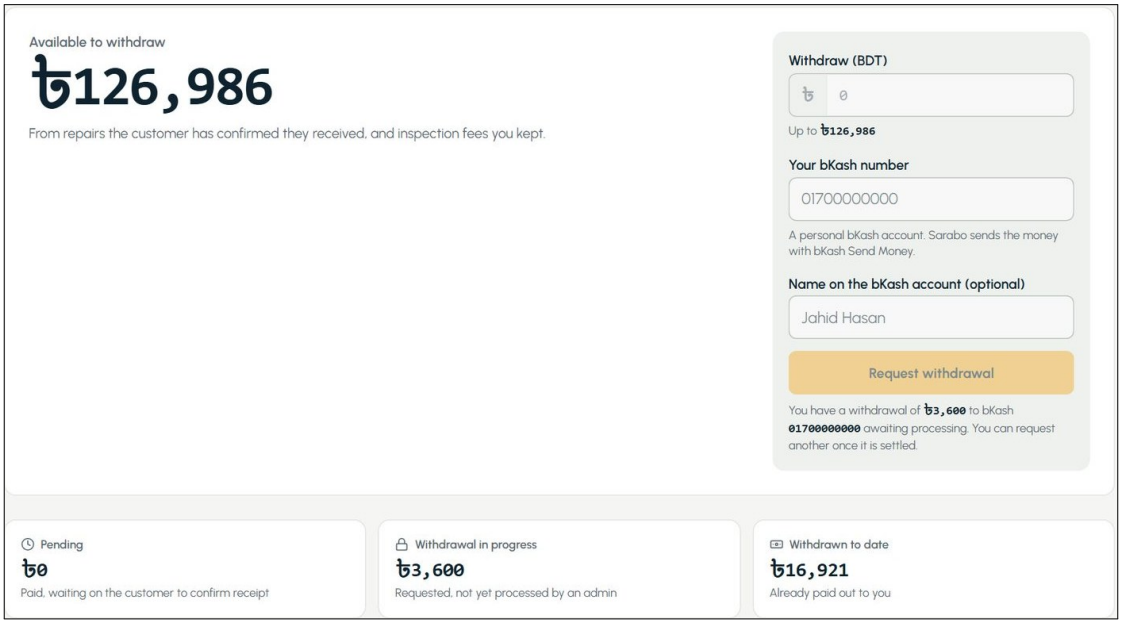


Figure 7.18: Technician wallet: the 3,600 BDT earning requested as a bKash withdrawal.

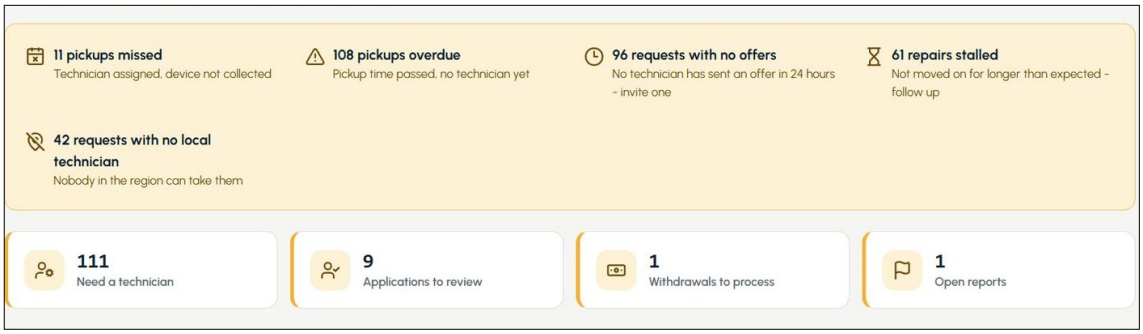


Figure 7.19: Admin overview: missed pickups, requests without offers and stalled repairs come first.

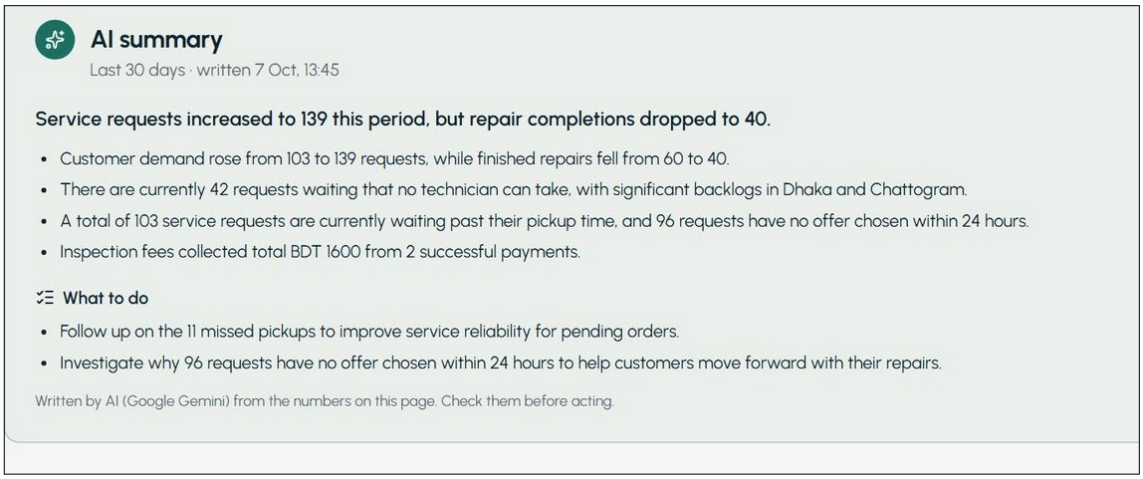


Figure 7.20: Admin reports with the AI summary; sentences with numbers not in the data are removed.

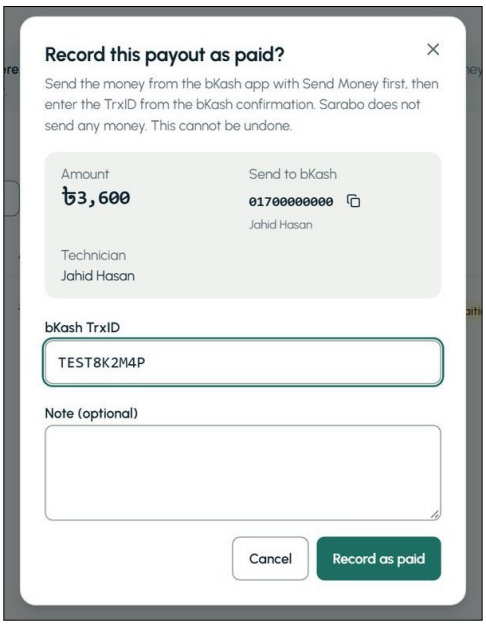


Figure 7.21: Recording a bKash payout with its TrxID (a test value).

Chapter 8

AI-Assisted Features

Sarabo uses Google Gemini in four places. In each the AI *suggests* and a person *decides*: nothing it returns changes a price, payment or request state without human review. All four are off by default and need an explicit setting and an API key; the client asks `GET /ai/status` and hides unavailable features. The model is `gemini-3.5-flash-lite`, with `gemini-3.1-flash-lite` as fallback when it is slow, overloaded or rate-limited.

8.1 AI Customer Support Chat

A chat on every page answers questions about how Sarabo works, its services and usual prices. The prompt is built from the *live* catalogue (`supportKnowledgeService`), so prices are always current. History, message and answer length are capped, and follow-up buttons (View services, Track a request, Log in, Create an account) come from a fixed list, not generated links (Figure 7.2).

8.2 AI Request Helper

On the request form a customer can describe the problem in Bangla, English or mixed "Banglish" and/or add a damage photo. The AI drafts the device, catalogue service, brand, model, an English description, a confidence level, an optional follow-up question and, for a photo, whether it shows a device. The form fills itself, the customer edits anything, and *Undo* restores the previous values (Figure 7.4). A photo showing a device is kept with the request's photos and sent when the

customer presses “Upload”, so it need not be picked twice.

8.3 Technician Price Guide and AI Offer Note

A price guide (computed without AI) shows the usual range from real offers for the same service in the last 180 days: the 25th percentile of “from” prices to the 75th percentile of “to” prices, and the same for fees; with fewer than three offers it uses the catalogue range. Offers far outside it get an advisory warning.

The technician can ask AI to write the offer note in English or Bangla. The model sees only what the technician sees (device, repair, problem text, whether photos exist), writes in the technician’s voice, presents the likely issue as a possibility, says the final price follows inspection, and uses no numbers. Phone numbers (including Bangla digits), emails, links and social handles are stripped, so contact stays on the platform (Figure 7.6).

8.4 AI Summary of Admin Reports

The Reports page (7, 30 or 90 days) can show an AI summary: a headline, two to four highlights and one to three actions. The model gets only aggregate figures (requests made, finished and cancelled, success rate, average days, refused demand by region, top devices and repairs, workload, inspection-fee money, attention counts), never names, emails or IDs. The last summary per period is stored, so reopening costs nothing (Figure 7.20).

8.5 Safeguards

Table 8.1 summarises how the main risks of LLM features [13] are handled.

All AI tests use fake providers, so the automated suites never call Gemini or spend money.

Table 8.1: AI risks and safeguards

Risk	Safeguard in Sarabo
Invented catalogue items	The request helper must answer with IDs from an enum built from the live catalogue (JSON schema); the server re-checks them and drops unknown or inactive IDs, or a repair belonging to another device.
Invented prices	No AI feature ever sets a price. Estimates come from the catalogue; the offer note contains no numbers.
Invented statistics	Every sentence of the report summary is checked: each number must appear in the facts, be a difference or rounded percentage between periods, a small count up to 10, or a fixed context number. Failing sentences are dropped; if nothing remains, the request fails with 502.
Prompt injection	Customer text is marked as untrusted data in the prompt, and output is forced into a schema.
Personal data leakage	Text containing a card number, password, token or tracking code is refused before any AI call. Models receive no names, emails or addresses. Contact details are stripped from generated notes.
Unbounded cost	Per-user 10-minute and daily limits plus a site-wide daily budget per feature, stored in MongoDB so all serverless instances share them. If the database is unreachable, the feature fails closed.
Bad photos	JPEG/PNG/WebP only, at most 1.5 MB, first bytes must match the type; the client shrinks photos to 1024 px first. The helper's photo is not stored by the AI endpoint.
Provider failure	If the first model is slow (over 5 s for support), overloaded or rate-limited, the request goes straight to the fallback model; otherwise controlled errors (503 unavailable, 504 timeout, 502 invalid answer) and the rest of the form keeps working. Live support answers take 1.4–1.9 s.

Table 8.2: Default AI usage limits

Feature	Per user / 10 min	Per user / day	Site / day
Support chat (signed in)	20	100	300
Support chat (anonymous)	8	30	(shared)
Request helper	10	30	200
Offer note	10	40	200
Report summary	5	20	50

Chapter 9

Security, Reliability and Data Integrity

9.1 Security Principles

Sarabo is *server-authoritative*: the client decides what to show, the server what is allowed, enforcing authorization at every resource operation [18]. Secrets (database, Stripe, Supabase, Gemini, Gmail, cron) live only in server environment variables, never in the repository or the browser.

9.2 Authentication

Each protected request carries a Firebase ID token, verified on the server with Firebase Admin; the caller's email comes only from that token, so endpoints such as `/technicians/me` or the wallet never take another user's email as input. Creating requests or payments needs a verified email address.

9.3 Role- and State-Based Authorization

Every write endpoint checks both who the caller is and what state the request is in:

- only Admins approve technicians, edit the catalogue, retry refunds, process withdrawals and moderate feedback;
- only an approved, matching Technician can see job details or send an offer, and only the booked Technician can inspect, quote, revise and repair;

- only the owning Customer can choose or reject offers, decide on a quote, pay, cancel, confirm receipt and review;
- a quote can be submitted only after inspection, payment only after approval, revision only from `quote_rejected`, and a paid repair can never re-enter the rejected-quote path.

Requests that do not belong to the caller return the same "not found" answer as missing ones, so their existence is not revealed.

9.4 Financial Integrity

Prices, fees, credits, commissions, balances and withdrawal limits are derived on the server from stored data, in whole taka. Stripe sessions use stored amounts, the webhook signature is checked on the raw body [10], payments and refunds are idempotent, settlements are frozen once, and a withdrawal moves to *paid* or *rejected* through a single-winner update, so it can never be paid twice.

9.5 Privacy

Public endpoints (tracking, jobs board, availability) return purpose-built projections with no names, emails, addresses, phone numbers, payment data or database IDs. Technicians see only the district and problem until booked; refused-demand counts store a hash, not the email; reports about technicians are admin-only.

9.6 Private Image Security

The bucket is private; signed URLs are short-lived and per object; the server checks the real stored type and size, and never returns the permanent object identifier.

9.7 Input Validation and Safe Errors

The server validates IDs, transitions, amounts, text lengths, file types and each mutation's allowed fields, rejecting anything unexpected; user text is stored and shown as plain text. Errors

use stable codes (e.g. `PICKUP_SLOT_FULL`) instead of raw provider messages, and logs keep only request ID, route, status and a category, never bodies or secrets.

9.8 CORS, Rate Limiting and Abuse Control

The API accepts browser calls only from the configured `https` client origin. Tracking, the jobs board and all AI features are rate limited, with AI limits stored in MongoDB so they hold across serverless instances; the scheduled job refuses to run without its secret.

9.9 Transaction Safety

MongoDB transactions protect the invariants that matter most:

- a request's state and its technician's availability always agree, so a technician cannot be double-booked;
- a payment, the booking it causes and the offers it closes change together;
- a quote transition, its tracking event and its notification apply together or not at all;
- a fee refund is marked pending in the same transaction as the event that causes it.

A late defect (Appendix E) showed a subtle rule: a duplicate-key error inside a transaction has already aborted it, so it must be passed on for the driver to retry, never swallowed.

9.10 Environment Isolation

Development, tests and production use different databases, and tests refuse to run against development or production. Seed and migration scripts default to dry runs and need explicit one-time flags for production; CI uses a throwaway database and no real credentials.

9.11 Legacy Compatibility Boundary

Some stored values come from the original parcel-delivery codebase (the role `rider`, statuses such as `driver_assigned` or `parcel_delivered`). They are frozen contract values behind a

presentation layer, so users only see repair terms; renaming them needs a data migration (future work).

Chapter 10

Testing and Evaluation

10.1 Testing Strategy

Testing focused on *domain correctness*: whether the right person can do the right thing at the right time, exactly once. Four layers were used (Figure 10.1).

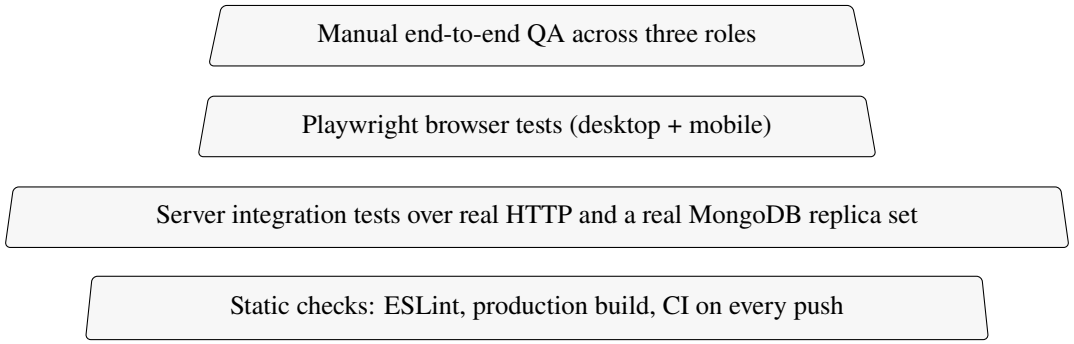


Figure 10.1: Testing layers used in Sarabo

Server integration tests. A custom harness (`test-all.js`) starts its own API on a random port against an isolated test database, seeds role accounts, an approved technician and the catalogue, and calls endpoints over real HTTP, plus direct service checks. Stripe, Supabase, Gemini and email are injected fakes, so no external service or money is touched. Races are tested by firing competing requests at once (two admins assigning, a cancellation racing a payment, one webhook delivered several times): each must give exactly one winner and a consistent state.

Browser tests. Playwright runs a production build of the client against a faked API on desktop and mobile (Pixel 7) viewports, covering public pages, the jobs board (listing, filters, sorting, empty and error states), tracking (valid, unknown and invalid codes, server errors), sign-in forms and route protection.

Continuous integration. GitHub Actions runs the server suites on a throwaway MongoDB 8 replica set, and client lint, build and browser tests, on every push and pull request. On the final commits (server 16 October, client 14 October 2026) the server workflow passed and the client lint-and-build job passed; the client browser-test job failed once on GitHub while the same 58 tests pass locally on that commit (Table 10.1).

10.2 Verification Results

For this report every check was run on 14–16 October 2026 against the final code (client commit `e9caaeb`, server commit `d81fa65`). The server suites were run against a temporary in-memory MongoDB replica set, as in CI, so no shared database state can affect the result. Table 10.1 shows the results.

Table 10.1: Verification results (14–16 October 2026)

Check	Passed	Failed
Server full suite (`npm test`)	2,761	0
of which re-run on their own: support and AI suite	164	0
of which re-run on their own: data-migration suite	63	0
of which re-run on their own: feedback suite	177	0
Client browser tests (Playwright, 29 scenarios × 2 viewports)	58	0
Client ESLint	no errors	
Client production build	success	
GitHub Actions, server, latest `main` run	success	
GitHub Actions, client lint and build	success	
GitHub Actions, client browser tests (same tests pass locally)	failed once	

The server suite has grown from 1,890 checks at the time of the earlier report (August 2026) to 2,761. The final phase added 181 of them: 15 for the final payout on removal, 4 for the AI fallback, 22 for offer ranking, 45 for the warranty, 72 for the no-stuck workflow fixes, 22 for profile editing and the minimum inspection fee, and 1 for the Stripe item name.

10.3 Representative Test Scenarios

Table 10.2: Representative test scenarios

ID	Scenario	Expected and verified result
T-01	Request without a token to a protected endpoint	401; no data returned
T-02	Customer calls an admin endpoint	403; nothing changes
T-03	Request in a region with no matching technician	409 `NO_TECHNICIAN_AVAILABLE`; refusal counted once per day
T-04	Customer picks a pickup slot that is already full	409 `PICKUP_SLOT_FULL`; request not saved
T-05	Unapproved or non-matching technician sends an offer	Refused; offer not stored
T-06	Inspection fee above the catalogue cap	Refused by the server
T-07	Fee payment arrives after another technician was booked	Payment recorded and refunded automatically
T-08	Same Stripe event delivered twice, or webhook and success page both fire	Exactly one payment row and one booking
T-09	Customer rejects an offer	Job reposted; technician may send one revised offer only
T-10	Cancel 3 hours vs 1 hour before pickup	Full refund vs fee kept and split 90/10
T-11	Stripe refund call fails	Refund stays pending, shows on admin home, retry succeeds once
T-12	Quote submitted before inspection	Controlled conflict; state unchanged
T-13	Quote rejected, revised and resubmitted	Old quote archived; new round gets its own notification; no transaction hang
T-14	Approved quote with fee already paid	Amount due = quote total minus fee; settlement on the full quote
T-15	Paid request moves on to repair states	Still reported as paid; no second Pay button
T-16	Repair completed but not yet received	Settlement pending; cannot be withdrawn
T-17	Withdrawal above available balance, or two open withdrawals	Refused; database allows only one open request
T-18	Withdrawal marked paid, then technician asks again	Nothing left to withdraw; balance can never go negative
T-19	Duplicate review, or review before receipt	Refused; one review per repair
T-20	AI draft names an unknown or inactive service	ID dropped by the server
T-21	AI summary sentence with a number not in the facts	Sentence removed before saving
T-22	Problem text containing a card number	Refused before any AI call
T-23	Public tracking and jobs board responses	No names, emails, addresses or payment data

ID	Scenario	Expected and verified result
T-24	Seed or migration script pointed at production	Refused without explicit operator override
T-25	Admin removes a technician with 3,000 BDT available	Removal and a 3,000 BDT final bKash withdrawal commit together; with no bKash number on record, removal is refused and nothing changes
T-26	Visitor searches "Ambarkhana" on the Service Areas page	Map zooms to Ambarkhana in Sylhet, not the district centre; "Sylhet" still opens the district
T-27	Two offers: nearby, quick, reliable technician vs. a cheaper one with a missed pickup	The first is ranked "Best match"; no private details in the response
T-28	Warranty claim covered, re-repair fails	Admin refunds the whole quote once; settlement reversed; already-withdrawn money shown as owed
T-29	Warranty claim after 30 days, or a third claim	Refused
T-30	Approved quote unpaid for 3 days, with the payment page open	Skipped that day (no one charged for a closed request); closed the next day, fee kept 90/10
T-31	Customer neither confirms nor disputes receipt	Confirmed automatically after 3 days; money released exactly once
T-32	Repair payment arrives after the request was cancelled	Payment recorded and refunded automatically
T-33	Offer with an inspection fee below the minimum	Refused (`INSPECTION_FEE_TOO_LOW`); an older low offer is charged at the minimum
T-34	Technician changes service area during an active repair	Refused; allowed once the repair is finished

10.4 Manual End-to-End Acceptance

In August 2026 the core lifecycle (request, assign, inspect, quote, reject, revise, approve, pay, repair, complete, confirm receipt) was accepted by hand with three accounts and Stripe test cards, checking that all roles saw the same state and that money became available only after receipt. On 14 October 2026 the marketplace flow was walked through the same way with real customer, technician and admin accounts: AI-assisted request, offer with AI note, inspection-fee payment, pickup condition, inspection, quote, approve-and-pay, repair, hand-back, review, a warranty claim to resolution, and a bKash withdrawal marked paid. Chapter 7 is the record of that run.

10.5 Evaluation Against Objectives

Table 10.3: Evaluation against objectives

Obj.	Status	Evidence
O1	Achieved	Firebase identity, server role and state checks (T-01, T-02)
O2	Achieved	Structured V2 form, catalogue, photos, pickup slots, AI helper (T-04, T-20)
O3	Achieved	Region and expertise gate, jobs board, offers, reposting and ranking (T-03, T-05, T-09, T-27)
O4	Achieved	Fee booking, credit, refunds and split (T-06, T-07, T-10, T-11, T-14)
O5	Achieved	Inspection, quote, rejection and revision (T-12, T-13)
O6	Achieved	Single payment truth, idempotent processing, PDF receipts (T-08, T-15)
O7	Achieved	Progress, evidence, receipt confirmation, reviews and reports (T-16, T-19)
O8	Achieved	47 notification types, email, daily reminders, sanitized tracking (T-23)
O9	Achieved	90/10 settlement, computed wallet, bKash withdrawals, final payout on removal (T-16 – T-18, T-25)
O10	Achieved	Attention list, reports with checked AI summary, catalogue editor, moderation (T-21)
O11	Achieved	2,761 server checks, 58 browser tests, lint, build, CI, live deployment
O12	Achieved	Warranty, deadlines and stalled-work alerts (T-28 – T-32)

10.6 Validation Boundary

The automated results come from an isolated environment with fake providers. Payments use Stripe test mode, so no real money has moved, and there are no real customers yet. Load testing and a formal security audit were out of scope.

Chapter 11

Limitations and Future Work

11.1 Current Limitations

- **Manual technician payouts.** The wallet tracks earnings and withdrawals exactly, but an admin sends the money by bKash Send Money and records the TrxID; there is no automatic payout through a bKash, Nagad or bank API.
- **Card payments in test mode.** Stripe has no local accounts for businesses in Bangladesh, so a launch needs a local gateway (SSLCOMMERZ or bKash Payment Gateway) behind the same payment service interface.
- **Pickup capacity checked, not locked.** Two customers booking the last place in a slot in the same second could both succeed. This is acceptable at the current scale (admins see every pickup time), but a busy service needs an atomic slot counter.
- **Provisional prices.** The 45 services and their ranges are a demonstration starting point, not prices approved by a real repair business; admins can change them at any time.
- **One active job per technician.** This prevents double booking but does not model technicians who handle several small jobs a day.
- **Legacy internal values.** Stored values such as `rider` and `parcel_delivered` come from the parcel-delivery codebase; they are hidden from users, but a migration would make the model cleaner.

- **Signed-in screens not in browser tests.** Browser tests cover public pages only, since sign-in needs real Firebase accounts. Signed-in screens are covered by server tests of every endpoint behind them and were checked by hand (Chapter 7).
- **No real customers yet.** The platform is deployed and tested, but has not run with real customers, technicians and money, and has had no load test or external security audit.

11.2 Future Enhancements

1. **Local payment gateway and automated payouts**: SSLCOMMERZ or bKash for customer payments and technician disbursement, with reconciliation.
2. **Live technician location**: the technician's approach on a map on pickup day.
3. **Mobile app or Progressive Web App**: an offline-friendly technician app with push notifications.
4. **SMS notifications**: many customers in Bangladesh rely on SMS more than email.
5. **Parts inventory**: parts used per repair, with cost and supplier.
6. **Technician scheduling**: daily capacity, working hours and route-aware assignment.
7. **Cash payment**: cash paid at hand-over, with the platform's share settled from the wallet.
8. **Own domain name**: a `.com` or `.com.bd` address, since some mobile networks failed to resolve the free `.vercel.app` address.
9. **Bangla interface**: full localisation of the interface, not only of AI input.
10. **Schema clean-up**: retire the legacy values through a planned migration.

Chapter 12

Conclusion

This project developed Sarabo, a platform that turns informal, trust-based electronics repair into a tracked, priced and auditable service. Customers describe a problem in their own language, compare ranked offers from qualified local technicians, pay an inspection fee that counts toward the final price, approve an itemised quote, follow every step and are protected by a 30-day warranty. Technicians get a steady source of matching jobs and a transparent wallet; admins get one view of everything that needs attention.

The main engineering outcome is a server-authoritative workflow: an action succeeds only if the caller has the right role, the request is in the right state and every amount is derived from stored data. Transactions, idempotent payments and refunds, and computed balances keep money and state consistent under retries and races, and every waiting state now has a deadline, an automatic step or an admin alert, so no repair can get stuck. Generative AI is used only as an assistant whose output is validated and reviewed by a person.

Since the earlier report Sarabo grew from an admin-assigned workflow into a live two-sided marketplace, and its automated server checks grew from 1,890 to 2,761, all passing, alongside 58 browser tests. The remaining limitations (Chapter 11), chiefly a local payment gateway, automated payouts and use by real customers, mark the path from a validated academic system to a live service.

References

[1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[2] Meta Platforms, Inc., “React documentation,” https://react.dev/, accessed: Oct. 2026.

[3] TanStack, “Tanstack query documentation,” https://tanstack.com/query/latest, accessed: Oct. 2026.

[4] Vite Contributors, “Vite documentation,” https://vite.dev/, accessed: Oct. 2026.

[5] Tailwind Labs, “Tailwind CSS documentation,” https://tailwindcss.com/docs, accessed: Oct. 2026.

[6] OpenJS Foundation, “Express documentation,” https://expressjs.com/, accessed: Oct. 2026.

[7] MongoDB, Inc., “MongoDB manual,” https://www.mongodb.com/docs/manual/, accessed: Oct. 2026.

[8] Google, “Firebase authentication documentation,” https://firebase.google.com/docs/auth, accessed: Oct. 2026.

[9] Stripe, Inc., “Stripe payments and checkout documentation,” https://docs.stripe.com/, accessed: Oct. 2026.

[10] ——, “Receive stripe events in your webhook endpoint,” https://docs.stripe.com/webhooks, accessed: Oct. 2026.

[11] Supabase, “Supabase storage documentation,” https://supabase.com/docs/guides/storage, accessed: Oct. 2026.

[12] Google, “Gemini API documentation,” https://ai.google.dev/gemini-api/docs, accessed: Oct. 2026.

[13] OWASP Foundation, “OWASP top 10 for large language model applications,” https://genai.owasp.org/llm-top-10/, accessed: Oct. 2026.

[14] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.2,” W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[15] Sheba Platform Limited, “Sheba.xyz,” https://www.sheba.xyz/, accessed: Oct. 2026.

[16] Urban Company, “Urban company,” https://www.urbancompany.com/, accessed: Oct. 2026.

[17] Thumbtack, Inc., “Thumbtack,” https://www.thumbtack.com/, accessed: Oct. 2026.

[18] OWASP Foundation, “REST security cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html, accessed: Oct. 2026.

Appendices

Appendix A

API Surface

All routes except the public ones require a Firebase ID token. "Role" shows who may call the route (C = Customer, T = Technician, A = Admin, P = public).

Table A.1: Sarabo API endpoints (main families)

Method	Route	Role	Purpose
Public			
GET	`/service-definitions`, `/service-definitions/:id`	P	Active catalogue
GET	`/service-availability`	P	Can a request be served here?
GET	`/pickup-slots`	P	Available pickup windows
GET	`/jobs`	P	Open jobs board
GET	`/public/trackings/:trackingCode`	P	Sanitized tracking
GET	`/ai/status`	P	Which AI features are on
POST	`/stripe-webhook`	Stripe	Signed payment events
GET	`/health`	P	Health check
Repair requests			
GET/POST	`/repair-requests`	C	List own / create
GET	`/repair-requests/:id`	C,T,A	Authorized detail
PATCH	`/repair-requests/:id/pickup-slot`	C	Reschedule pickup
PATCH	`/repair-requests/:id/cancel`	C	Cancel before booking
GET/POST	`/repair-requests/:id/booking-cancellation`, `/cancel-booking`	C	Preview / cancel a booked visit
POST	`/ai/request-draft`	C	AI request helper
Job portal			
GET	`/technician/job-portal`	T	Matching open jobs with details
POST	`/repair-requests/:id/applications`	T	Send an offer
GET	`/repair-requests/:id/applications/price-guide`	T	Price guide
POST	`/repair-requests/:id/applications/ai-note`	T	AI offer note
POST	`/repair-requests/:id/applications/mine/withdraw`	T	Withdraw own offer

Method	Route	Role	Purpose
POST	/repair-requests/:id/applications/:applicationId/accept	C	Choose an offer
POST	/repair-requests/:id/applications/:applicationId/decline	C	Reject an offer (repost)
Inspection, quote, payment, repair			
POST	/repair-requests/:id/inspection	T	Submit inspection
POST	/repair-requests/:id/quote	T	Submit quote
POST	/repair-requests/:id/quote/decision	C	Approve / reject
POST	/repair-requests/:id/quote/revise	T	Revise rejected quote
POST	/repair-requests/:id/quote/cancel-request, /quote/close	T, C	End after a declined quote
GET	/repair-requests/:id/payment-eligibility	C	Can this be paid now?
POST	/repair-requests/:id/checkout-session	C	Start repair payment
GET	/payments, /payments/:transactionId/receipt.pdf	C,A	History and PDF receipt
POST	/repair-requests/:id/repair/start, /progress, /evidence, /complete	T	Repair workflow
POST	/repair-requests/:id/confirm-receipt	C	Confirm hand-over
Technicians, wallet and feedback			
GET	/technicians/me, /technicians/me/reviews	T	Own profile and reviews
PATCH	/technicians/:id/expertise	T,A	Edit expertise
GET	/technician/wallet	T	Wallet balances
POST	/technician/withdrawals	T	Request bKash withdrawal
GET	/repair-requests/:id/feedback	C	Review eligibility and own feedback
POST	/repair-requests/:id/review, /technician-reports	C	Review / private report
GET	/notifications, /notifications/unread-count	C,T,A	In-app notifications
Administration			
GET	/admin/repair-requests	A	Paginated, filterable requests
POST	/admin/repair-requests/:id/invite	A	Invite a technician to apply
POST	/admin/repair-requests/:id/missed-pickup/withdraw	A	Technician no-show
POST	/admin/repair-requests/:id/inspection-fee/retry-refund	A	Retry a pending refund
GET	/admin/attention, /admin/reports	A	Attention list and reports
GET/POST	/admin/reports/ai-summary	A	AI summary
GET/PATCH	/admin/service-definitions, /:id	A	Catalogue editor
PATCH	/technicians/:id	A	Approve / reject technician
GET/POST	/admin/withdrawals, /:id/mark-paid, /:id/reject	A	Withdrawals
GET/PATCH	/admin/technician-reviews, /admin/technician-reports	A	Moderation
PATCH	/users/:id/role	A	Change a user's role
PATCH	/technicians/me	T	Edit own profile and service area
GET	/repair-requests/:id/applications	C,A	Offers with ranking (match)
GET/POST	/repair-requests/:id/warranty, /warranty-claims, /warranty/photos	C	Warranty status, claim, photos
POST	/warranty-claims/:claimId/confirm, /review-request	C	"It works now" / ask admin review
GET/POST	/technician/warranty-claims, /warranty-claims/:claimId/schedule, /decision, /rerepair	T	Re-inspection and re-repair

Method	Route	Role	Purpose
GET/POST	`/admin/warranty-claims`, `/:claimId/review`, `/decision`, `/cannot-fix`, `/refund`	A	Warranty review, take-over, refund
POST	`/repair-requests/:id/cancel-unpaid`, `/close-not-repairable`	C,T	End an unpaid or unrepairable request
POST	`/repair-requests/:id/repair/cannot-complete`, `/mark-returned`, `/customer-not-available`	T	Workflow exceptions
POST	`/repair-requests/:id/receipt-dispute`, `/confirm-return`, `/pickup-condition/dispute`	C	Disputes and return confirmation
POST	`/admin/repair-requests/:id/cancel-paid-repair`, `/confirm-receipt`, `/pickup-condition/reviewed`	A	Admin resolution
GET	`/cron/pickup-reminders`	Cron	Daily reminders, deadlines and stalled-work alerts

Appendix B

Data Model Summary

Table B.1: Selected fields of a repair request

Field	Meaning
`schemaVersion`	2 for modern requests
`product`	Device category, brand and model snapshot
`service`	Chosen service with its price rule snapshot
`trackingId`	Public tracking code (`SRB-...`)
`deliveryStatus`	Lifecycle state (frozen legacy name)
`pickupSlot`	Chosen two-hour window and region
`damage.images`	Up to three private image identifiers
`repostedAt`, `repostCount`	Job-board reposting after a rejected offer
`technicianId`, `technicianEmail`	Booked technician
`assignmentHistory`	Append-only assignment audit trail
`inspectionPayment`	Inspection fee payment and refund state
`inspectionFeeSettlement`	90/10 split of a kept fee
`inspection`	Findings and detected issues
`quote`, `quoteHistory`	Live quote and archived rejected quotes
`payment`	Repair payment state
`repair`	Progress, completion and evidence
`technicianSettlement`	Frozen commission and receivable
`closedBy`	Who ended a declined request
`pickupCondition`	Device condition and accessories recorded at pickup, with any dispute and admin review
`deviceReturn`	Hand-back of the device (after repair or cancellation): marked, disputed, resolved

Appendix C

Financial Rules

$$\text{Repair subtotal} = \text{Labour} + \text{Parts} + \text{Additional charges}$$

$$\text{Amount due} = \max(\text{Quote total} - \text{Inspection fee paid},\ 0)$$

$$\text{Sarabo commission} = \text{round}(10\% \times \text{Repair subtotal})$$

$$\text{Technician receivable} = \text{Repair subtotal} - \text{Sarabo commission}$$

$$\text{Kept inspection fee} : \text{same 90/10 rule, whole taka}$$

$$\text{Minimum inspection fee} = 100 \text{ if the highest estimate} \leq 1{,}000,\ \text{else } 200$$

$$\text{Warranty refund} = \text{Quote total (repair payment + credited fee); settlement reversed}$$

All amounts are whole Bangladeshi taka (BDT). The settlement is calculated on the full quote, even when part of it was covered by the inspection fee. A settlement becomes withdrawable only after the customer confirms receipt; a kept inspection-fee share is available immediately.

Table C.1: Worked example

Inspection fee paid at booking	BDT 300
Approved quote (labour 1,200 + parts 1,500 + other 0)	BDT 2,700
Customer pays after approval	BDT 2,400
Sarabo commission (10% of 2,700)	BDT 270
Technician receivable	BDT 2,430

Appendix D

Source and Build Notes

Repositories. Client: `https://github.com/jahidjubaer/sarabo-client`; server: `https://github.com/jahidjubaer/sarabo-server`.

Deployment. Client: `https://sarabo-client-six.vercel.app`; API: `https://sarabo-server.vercel.app`.

Table D.1: Verified code baseline

Item	State
Client branch / commit	`main` / `e9caaeb115581050c2887f828384447d5301c8fa`
Server branch / commit	`main` / `d81fa65c40723ad38ee0663a6821a9459798e4bf`
Commits	163 (client), 120 (server), 26 Jul – 16 Oct 2026
Size	about 337 client source files (≈29,000 lines); about 191 server files (≈45,600 lines, including tests)
Server tests	2,761 / 2,761 passed
Browser tests	58 / 58 passed
Lint and build	Passed

Listing D.1: Development commands

```
# client (sarabo-client)
npm install
npm run dev # http://localhost:5173
npm run lint
npm run build
npm run test:e2e # Playwright browser tests
```

```
# server (sarabo-server)
npm install
npm run dev # http://localhost:3000
npm test # full suite, isolated test database
npm run seed:service-definitions -- --confirm-seed
```

Configuration. No secret values appear in this report. Database, Firebase, Stripe, Supabase, Gemini, Gmail and cron credentials are supplied through environment variables, documented by name in each repository's `.env.example`, `README.md` and `SETUP.md`.

Appendix E

Incident Record: Revised Quote Hang

Observed failure. During manual testing in August 2026, submitting a *revised* quote (after the customer had rejected the first one) left the technician's screen on "Working..." indefinitely.

Cause. The "quote submitted" notification used a de-duplication key that was the same for the whole life of a request. The second round therefore hit a duplicate-key error inside the database transaction. The code treated that error as a harmless duplicate and carried on, but MongoDB had already aborted the transaction. The commit then failed, the driver retried the same callback, and the cycle repeated until it timed out, with nothing ever saved.

Fix. The key now includes the quote round, derived on the server:

Listing E.1: Round-aware quote notification key

```
round = quoteHistory.length + 1
key = "repair:" + requestId + ":quote_submitted:" + round
```

In addition, notification creation inside a transaction first checks for an existing key in the same session, and if a race still causes a duplicate-key error, the error is passed on so that the driver restarts the whole transaction cleanly instead of continuing an aborted one. Regression tests cover first and revised submissions, same-round replays and transactional races.

Lesson. Inside a database transaction, an error may already have invalidated the transaction by the time application code sees it. Such errors must be propagated, not swallowed.